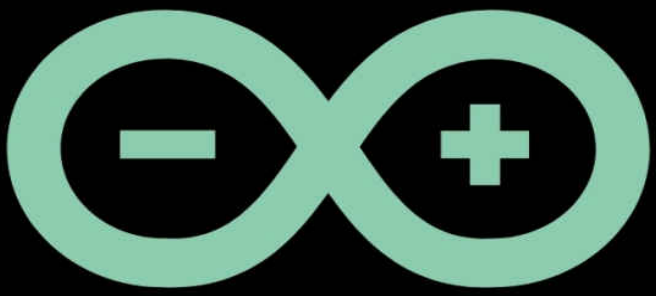
MEM LNC

Arduino

Manuel pour apprendre a programmer Arduino



Troisième édition
2021-5-9

& by Emma William
Claudia Alves

Arduino

Manuel pour apprendre a programmer Arduino

2020

3 nd édition

Par Claudia Alve s

mEmlnc

«La programmation ne concerne pas ce que vous savez, mais ce que vous pouvez comprendre . » -

Chris Pine

Ce livre vous aidera à développer un code source fonctionnel pour le microcontrôleur Arduino. Dans ces pages, nous nous intéresserons principalement à l'aspect logiciel de l'informatique physique - la conception de code pour travailler avec des objets physiques qui présentent un comportement ou une interactivité via le logiciel. En commençant par le contexte de base duPlateforme Arduino pour être opérationnel avec notre premier code, nous discuterons de la

structure et de la syntaxe du langage de programmation basé sur C d'Arduino, en examinant les variables, les structures de contrôle, les tableaux et la mémoire. Ce livre abordera ensuite de nombreuses fonctions uniques au développement Arduino pour contrôler les entrées et sorties numériques et analogiques, la synchronisation, le caractère aléatoire, les fonctions d'écriture et l'utilisation de nombreuses bibliothèques Arduino pour travailler avec différents types de matériel et de protocoles de communication. Arduino, comme Processing avant lui, a adopté l'idée d'un carnet de croquis de code. Nous poursuivrons cette métaphore en évoquant le processus d'esquisse dans le code en tant que méthode intuitive pour tester rapidement de nouvelles idées dans le code. La plupart de ce livre est écrit autour de cette idée de développer des compétences en programmation par le biais du dessin. Nous vous proposerons également des suggestions de nouveaux projets et matériels, de nouveaux langages à essayer et des moyens de contribuer à la communauté. Ce livre ne s'attarde pas trop longtemps sur la théorie de l'électronique, la conception de circuits, le piratage ou d'autres pratiques spécifiquement basées sur le matériel, bien que nous reviendrons sur le côté matériel des choses dans notre dernier chapitre pour fournir une petite base pour l'informatique physique. Ce livre reprend à bien des égards là où le cahier de programmation Arduino s'est arrêté, avec des discussions encore plus approfondies sur l'environnement Arduino; exemples de code simples et sans fioritures; et des schémas clairs et faciles à lire etLe cahier, un petit livret PDF, a été ma première expérience d'écriture sur l'Arduino et n'a jamais été censé être plus qu'un bref guide pour mes étudiants lorsque j'ai introduit pour la première fois une classe de 15 majors universitaires en art et design à l'Arduino en 2007 . Les meilleurs plans et tout, ce petit livret a maintenant été traduit en espagnol, russe et néerlandais (que je connais), est hébergé dans tellement d'endroits différents qu'il est impossible de suivre, et il a été utilisé dans des ateliers et classes à travers le monde. Je n'ai pas mis à jour le cahier au cours des dernières années et, en toute honnêteté, je ne sais pas trop quoi en faire maintenant, alors j'espère que ce nouveau livre comblera un vide et

trouvera une adoption similaire et généralisée que le petit livret a apprécié toutes ces années.

Lcd . print (" Traduction de John Bach ");

introduction

Environnement de codage Arduino et outils de base

Quelle langue est Arduino?
IDE Arduino
Chapitre un
Transactions en langage Arduino
Transactions arithmétiques dans Arduino
Transactions budgétaires dans Arduino
Transactions booléennes dans Arduino
Indexer les transactions d'accès dans Arduino
Coefficients de nombre binaire dans Arduino
Transactions composées dans Arduino

Chapitre II
Types de données
L'objet String dans l'Arduino
Structure générale
Valeurs renvoyées
Les serviteurs
Transactions
Baies dans Arduino
Créer un tableau
Accès à la matrice
Tapez bool dans Arduino
octet dans Arduino
Tapez char dans Arduino
Tapez double dans Arduino
Tapez float dans Arduino
Structure générale
Tapez int dans Arduino
Cordes dans Arduino
Structure générale
Caractère de terminaison nul

Emballage de longues cordes
Matrices de chaînes de texte
Tapez un mot dans Arduino
Le mot-clé void dans Arduino

Chapitre III
Constantes et variables
Système décimal (base 10)
Système binaire (base 2)
Notes et avertissements
Constantes dans Arduino
Portée variable et qualificatifs
Le mot-clé const dans Arduino
Le domaine des variables dans Arduino
Le mot-clé statique dans Arduino
Exemples
Mot-clé volatile dans Arduino
Entiers volatils ou longs?

Chapitre IV
Structures de contrôle
Rupture de mot-clé dans Arduino
Continuer le mot-clé dans Arduino
Bouclez ... tout en itératif dans Arduino
Mot-clé else dans Arduino
Aller à mot-clé dans Arduino
Expression si conditionnelle dans Arduino
Notes et avertissements
Retour de mot-clé dans Arduino
Exemples
Boîtier de commutateur d'expression en Arduino

Transactions
Alors que l'épisode itératif dans Arduino

Chapitre V
Les fonctions
La fonction digitalRead () dans l'Arduino
digitalRead (broche)
La fonction digitalWrite () dans l'Arduino
La fonction pinMode () dans l'Arduino
Fonctions de temps dans Arduino
La fonction delay () dans l'Arduino
Fonction Arduino delayMicrosecondes ()
Fonction Arduino micros ()
Fonction Arduino millis ()
Fonctions mathématiques dans Arduino
Manipulation et vérification des polices dans Arduino
Fonctions de conversion Arduino
Fonctions de contrôle provinciales dans Arduino

Chapitre VI
Outils supplémentaires
Le programme de mots clés dans Arduino
Exemples

Notes et avertissements

introduction

Depuis le lancement de la plateforme open-source Arduino, la marque s'est imposée au centre d'une vaste communauté open-source. L'écosystème Arduino est composé d'une combinaison diversifiée de matériel et de logiciels. La polyvalence d'Arduino et son interface simple en font un choix de premier plan pour un large éventail d'utilisateurs à travers le monde, des amateurs, concepteurs et artistes aux prototypes de produits.

La carte Arduino est connectée à un ordinateur via USB, où elle se connecte à l'environnement de développement Arduino (IDE). L'utilisateur écrit le code Arduino dans l'IDE, puis le télécharge sur le microcontrôleur qui exécute le code, interagissant avec les entrées et les sorties telles que les capteurs, les moteurs et les lumières.

Les débutants et les experts ont accès à une multitude de ressources et de matériel gratuits pour les soutenir. Les utilisateurs peuvent rechercher des informations sur la configuration de leur carte ou même sur la façon de coder sur Arduino. L'open source derrière Arduino l'a rendu particulièrement convivial pour les utilisateurs nouveaux et expérimentés. Il existe des milliers d'exemples de code Arduino disponibles en ligne. Dans ce livre, nous expliquerons tous les principes de base dont un programmeur novice a besoin pour entrer dans le monde du génie, Arduino

Environnement de codage Arduino et outils de base

Quelle langue est Arduino?
Le code Arduino est écrit en C ++ avec un ajout de méthodes et de fonctions spéciales, que nous mentionnerons plus tard. C ++ est un langage de programmation lisible par l'homme. Lorsque vous créez un 'sketch' (le nom donné aux fichiers de code Arduino), il est traité et compilé en langage machine.

IDE Arduino

L'environnement de développement intégré Arduino (IDE) est le principal programme d'édition de texte utilisé pour la programmation Arduino. C'est là que vous allez taper votre code avant de le télécharger sur le tableau que vous souhaitez programmer. Le code Arduino est appelé esquisses.

Arduino est une plate-forme open source utilisée pour construire des projets électroniques (allant de projets simples tels que des thermomètres à des projets complexes tels que des robots, des imprimantes 3D et des applications IoT). L'objectif principal de la création de la plate-forme Arduino est de fournir une plate-forme conviviale pour aider les personnes sans connaissances préalables en électronique et en programmation. La plate-forme Arduino se compose de deux sections principales: la section matérielle et la section logicielle. La section matérielle se compose d'une carte Arduino et des composants électroniques associés et d'autres composants matériels, tandis que la section logicielle se compose de l'environnement de développement Arduino (Arduino IDE), qui représente l'environnement hôte pour écrire le code dans le langage Arduino et le télécharger sur le Cartes Arduino pour contrôler la section matérielle.

Arduino est juste un ensemble de fonctions C / C ++, qui sont principalement dérivées de C et C ++ et des frameworks de câblage et de traitement et sont open source. Le langage Arduino est utilisé pour programmer des cartes Arduino de différents types. Arduino est distinct du langage dérivé du C, étant un langage orienté objet, contenant de nombreuses classes et objets tels que String et Stream. De plus, Arduino est riche en de nombreuses bibliothèques qui fournissent plus de fonctionnalités telles que travailler avec n'importe quelle pièce ou élément électronique supplémentaire, modifier des données, etc. Arduino ne doit pas seulement être utilisé avec l'IDE Arduino, mais peut également être utilisé avec un tiers - environnements de développement de partie tels qu'Eclipse.

Chapitre un

Transactions en langage Arduino

Transactions arithmétiques dans Arduino

Les opérateurs arithmétiques effectuent les quatre opérations arithmétiques de base (addition, soustraction, multiplication et division) en plus d'autres opérations telles que le calcul du reste de la division et l' attribution de valeurs aux variables.

Module%

Calcule le reste en divisant deux nombres l'un par l'autre.

Laboratoires *

Calcule le produit de la multiplication de deux nombres ensemble.

FONCTIONNEMENT +

Calcule le résultat de l'addition de deux nombres ensemble.

-

Calcule le résultat de la soustraction de deux nombres l'un de l'autre.

Laboratoires /

Calcule le résultat de la division de deux nombres par un.

Paramètre =

Le coefficient d'attribution indique au contrôleur d'évaluer l'expression ou la valeur du côté droit du coefficient = peu importe et de le stocker dans la variable du côté gauche de ce coefficient.

Transactions budgétaires dans Arduino

Les opérateurs de comparaison effectuent un processus d'équilibrage, comme on l'appelle, entre deux valeurs, deux variables, une variable et une valeur donnée, puis retournent une valeur logique qui représente l'état d'un opérateur (plus grand, plus petit, égal, etc.) relatif à l'autre.

Paramètre! =

Équilibrer deux valeurs ou deux variables et retourner la vraie valeur si elles ne sont pas égaux.

paramètre <

Équilibrer deux valeurs ou deux variables et retourner la valeur réelle si le coefficient sur le côté gauche est complètement plus petit que le coefficient sur le côté droit.

Paramètre <=

Balances deux ou deux valeurs et renvoie vrai si le coefficient du côté gauche est inférieur ou égal au coefficient sur le côté droit.

Paramètre ==

Équilibrez deux ou deux variables et renvoie vrai si le coefficient du côté gauche est égal au coefficient du côté droit.

Paramètre>

Équilibrer deux valeurs ou deux variables et retourner la valeur réelle si le coefficient sur le côté gauche est exactement plus grand que le coefficient sur le côté droit.

Paramètre> =

Un équilibre entre deux ou deux valeurs et renvoie vrai si le coefficient du côté gauche est supérieur ou égal au coefficient sur le côté droit.

Transactions booléennes dans Arduino

Les opérateurs booléens exécutent des opérateurs booléens (opération AND, opération OR ou opération NOT) sur des valeurs ou des expressions logiques et renvoient la valeur booléenne résultante.

!

La logique reflète la valeur ou l'expression logique qui la précède en lui appliquant l'opération logique NOT;

&&

Applique l'opération logique AND à deux valeurs ou expressions logiques et renvoie la valeur logique résultante

||

Applique l'opération OR logique à deux valeurs ou expressions logiques et renvoie la valeur logique résultante

Indexer les transactions d'accès dans Arduino

Pointeur d' accès Opérateurs (pointeur d' accès opérateurs) sont des transactions qui sont utilisées avec des curseurs pour permettre l' accès du curseur et des références à des adresses mémoire variables et les variables aux valeurs d'accès contenues dans les adresses mémoire référencées par les curseurs.

&

Il est utilisé pour fournir une référence à l'adresse mémoire de référence d'une variable. Une référence est une fonctionnalité importante utilisée avec les pointeurs.

*

Il permet d'accéder à une valeur stockée dans une adresse mémoire spécifique (Déréférencement) d'un indicateur. L'accès aux données

stockées dans les adresses mémoire est une fonctionnalité importante utilisée avec les pointeurs.

Coefficients de nombre binaire dans Arduino

Opérateurs de bits effectue toutes les opérations logiques et les décalages de bits de numéro et de valeurs binaires et renvoie la valeur résultante.

&

En C ++, l'opération AND est effectuée sur chacun des deux bits correspondants des nombres donnés indépendamment et donne la valeur numérique résultante.

>>

Les bits à sa gauche vers la gauche déplacent un nombre spécifié de chiffres égal à la valeur vers la droite.

<<

Les bits à gauche de celui-ci vers la droite déplacent un nombre spécifié de chiffres égal à la valeur de droite.

^

En C ++, l'opération XOR est effectuée sur chacun des deux bits correspondants des nombres donnés indépendamment et donne la valeur numérique résultante.

|

En C ++, le OU opère sur chacun des deux bits correspondants des deux nombres donnés indépendamment et donne la valeur numérique résultante.

~

En C ++, l'opération NOT est effectuée sur chacun des bits donnés indépendamment - contrairement au coefficient & & | - et donne la valeur numérique résultante.

Transactions composées dans Arduino

Les opérateurs composés sont fréquemment utilisés pour écrire des codes dans Arduino à zéro, réglage fin et d' inverser certains bits des valeurs binaires et d' autres processus importants qui simplifient grandement et l' écriture de code Simplify.

Parameter = &

Habituellement utilisé avec une variable constante pour mettre à zéro une valeur de bit spécifique (c'est-à-dire faire une valeur de 0 ou LOW).

Module = *

C'est un raccourci pour multiplier la valeur actuelle d'une variable par une constante ou une autre variable, puis stocker le résultat dans la même variable.

FONCTIONNEMENT ++

Augmente la valeur de la variable utilisée de 1.

Parameter = +

Un raccourci est le processus consistant à combiner la valeur actuelle d'une variable avec une constante ou une autre variable, puis à stocker le résultat dans la même variable.

-

Diminue la valeur de la variable utilisée de 1.

Module = -

Un raccourci pour une constante ou une autre variable est soustrait de la valeur actuelle d'une variable et le résultat est stocké dans la variable elle-même.

Module = /

Un raccourci est un processus consistant à diviser la valeur actuelle d'une variable par une constante ou une autre variable, puis à stocker le résultat dans la même variable.

Module = ^

Habituellement utilisé avec une variable constante pour inverser (inverser) une valeur de bit spécifique.

Parameter = |

Il est généralement utilisé avec une variable de constante pour définir une valeur de bit spécifique (définie sur 1 ou HIGH).

Chapitre II

Types de données

L'objet String dans l'Arduino

La fonction String () crée une copie de la classe String. Il existe plusieurs façons de créer des chaînes de texte de différents types de données: appelez la fonction String () avec:

Chaîne de texte statique de caractères entre guillemets doubles (c'est-à-dire un tableau de caractères), ou

Un seul caractère constant entre guillemets simples, ou

Une autre copie de l'objet String, ou

Constante entière ou constante longue, ou

Un entier fixe ou un nombre fixe long utilisant une base spécifiée, ou

Nombre entier ou variable long, ou

Variable de nombre entier ou long utilisant une base spécifiée, ou

Un nombre décimal ou double décimal utilisant des décimales spécifiées.

Si vous appelez la fonction String () avec un nombre, elle crée une chaîne de texte contenant une représentation des nombres de ce nombre via des caractères ASCII. La base par défaut utilisée dans ce cas est 10. L'exemple suivant serait donné:

String thisString = String (13);

Chaîne de texte "13". Une base différente peut être utilisée telle que:

String thisString = String (13, HEX);

Vous donnerez la chaîne de texte "D" qui est la représentation hexadécimale de la valeur décimale de 13. Si vous préférez le binaire, essayez cet exemple:

String thisString = String (13, BIN);

Ce qui donnera la chaîne de texte "1101" qui est la représentation binaire de la valeur décimale de 13.

Structure générale

String (val)

String (val, base)

String (val, decimalPlaces)

Transactions

val

La variable ou la valeur à mettre en forme sous forme de chaîne de texte. Les types de données autorisés sont les suivants : chaîne de texte, police de caractères, octet, entier, entier inutile, nombre long sans valeur, nombre décimal et décimal multiplicateur.

base

Le coefficient facultatif détermine la base du système numérique dans lequel la valeur numérique sera formatée. La valeur par défaut est: DEC.

decimalPlaces

Limite le nombre de décimales (nombres après la virgule) à la décimale ou au multiple de la décimale.

Valeurs renvoyées

Une copie de la classe String contenant la valeur val est renvoyée.

Exemples

Tous les exemples suivants sont des exemples valides d'utilisation de la fonction String ():

String stringOne = "Bonjour String"; // Utiliser une chaîne de texte fixe

String stringOne = String ('a'); // Convertit une police fixe en une chaîne de texte

String stringTwo = String ("Ceci est une chaîne"); // String Convertit une chaîne de texte statique en un objet de type

String stringOne = String (stringTwo + "avec plus"); // Combine deux chaînes de texte ensemble

String stringOne = String (13); // Utiliser un entier constant

String stringOne = String (analogRead (0), DEC); // Utilisation d'un entier spécifiant la base utilisée

String stringOne = String (45, HEX); // "Utiliser un entier avec une base définie sur" hex

String stringOne = String (255, BIN); // "Utiliser un entier avec la ligne de base définie sur" binaire

String stringOne = String (millis (), DEC); // Utilisez un nombre long et spécifiez la base utilisée

String stringOne = String (5.698, 3); // Utilisez un nombre décimal et spécifiez le nombre de décimales

Les serviteurs

charAt ()

La fonction récupère un caractère avec un index spécifique à partir de la chaîne de texte qui est appelée avec elle.

compareTo ()

La fonction équilibre deux chaînes de texte et détermine laquelle est supérieure à l'autre ou si elles sont égales.

concat ()

La fonction ajoute le paramètre qui lui est passé à la chaîne de texte qui est appelée avec elle.

c_str ()

La fonction convertit le contenu de la chaîne de texte avec laquelle elle est appelée dans le style de chaîne de texte idéal en C (la chaîne de texte se terminant par null).

se termine avec ()

La fonction vérifie si la chaîne de texte appelée avec elle se termine par les mêmes caractères qui lui ont été transmis.

équivaut à ()

La fonction vérifie si la chaîne de texte que vous appelez est égale à la chaîne de texte que vous avez passée.

equalsIgnoreCase ()

La fonction vérifie si la chaîne que vous appelez est égale à la chaîne que vous avez passée, sans tenir compte de la casse.

getBytes ()

La fonction récupère un nombre spécifié d'octets de la chaîne de texte qui est appelée avec elle et la place dans un référentiel spécifique.

indexOf ()

L'index enfant renvoie la première apparition d'un caractère ou d'une chaîne de texte particulier dans la chaîne de texte qui est appelée avec lui.

lastIndexOf ()

L'index enfant renvoie la dernière apparition d'un caractère ou d'une chaîne de texte particulier dans la chaîne de texte qui a été appelée avec lui.

longueur ()

La fonction renvoie le nombre de caractères de la chaîne qui est appelée avec elle.

retirer ()

La fonction supprime un nombre spécifié de caractères de la chaîne de texte avec laquelle elle est appelée.

remplacer ()

L'enfant substitue une police ou une sous-chaîne là où une police ou une sous-chaîne spécifique existe dans la chaîne qui est appelée avec elle.

réserve ()

L'esclave réserve une partie de la mémoire et l'alloue pour manipuler et modifier la chaîne de texte qui est appelée avec lui.

setCharAt ()

La fonction affecte la valeur d'un caractère qui est positionné dans la chaîne de texte avec laquelle il est appelé.

commence avec ()

La fonction vérifie si la chaîne de texte appelée avec elle commence par les mêmes caractères qui lui ont été transmis.

sous-chaîne ()

L'enfant récupère une partie spécifique de la chaîne de texte qui est appelée avec elle.

toCharArray ()

La fonction convertit tout ou partie de la chaîne de texte qui est appelée avec elle en une chaîne de caractères.

toInt ()

La fonction convertit la chaîne de texte avec laquelle elle est appelée en un entier.

toFloat ()

La fonction convertit la chaîne de texte avec laquelle elle est appelée en décimal.

toLowerCase ()

La fonction convertit tous les caractères de la chaîne de texte avec laquelle elle est appelée en lettres minuscules.

toUpperCase ()

La fonction convertit tous les caractères de la chaîne de texte avec laquelle elle est appelée en majuscules.

couper ()

La fonction supprime tous les espaces blancs du début et de la fin de la chaîne de texte appelée avec elle.

Transactions

[]

L'opérateur donne accès et renvoie un caractère spécifique de la chaîne utilisée.

+

L'opérateur combine (fusionne) deux chaînes de texte ensemble en une nouvelle chaîne de texte, puis les renvoie.

= +

L'opérande ajoute tout type de données à la fin d'une chaîne de texte particulière.

==

Le coefficient vérifie que deux chaînes de texte sont égales l'une à l'autre.

<

L'opérande vérifie si la chaîne de texte à gauche est exactement plus grande que la chaîne de texte à droite.

= <

L'opérande vérifie si la chaîne de texte à gauche est supérieure ou égale à la chaîne de texte à droite.

>

L'opérande vérifie si la chaîne de texte à gauche est plus petite que la chaîne de texte à droite.

=>

L'opérande vérifie si la chaîne de texte à gauche est inférieure ou égale à la chaîne de texte à droite.

=!

Le paramètre vérifie si la chaîne de texte à gauche est différente de la chaîne de texte à droite.

Baies dans Arduino

Un tableau est un ensemble de variables placées dans un seul conteneur accessible par leur numéro d'index. Les matrices en C, sur lesquelles Ardonyo est basé, peuvent être un peu compliquées, mais les utiliser sous la forme la plus simple est facile.

Créer un tableau

Il existe plusieurs façons de créer un tableau:

int myInts [6];

int myPins [] = {2, 4, 8, 3, 6};

int mySensVals [6] = {2, 4, -8, 3, 2};

char message [6] = "bonjour";

Vous pouvez déclarer un tableau sans l' initialiser et de placer des valeurs en elle comme dans myInts . Un tableau peut être déclaré sans spécifier explicitement sa taille, comme myInts ; dans ce cas, la banque comptera les éléments et donnera à la matrice une taille appropriée.

Cependant, vous pouvez configurer un tableau et spécifier sa taille comme dans le cas du tableau mySensVals . Notez que lors de la déclaration d'un tableau de caractères, vous devez augmenter la taille du tableau à la taille requise pour accueillir un seul élément supplémentaire, le null, qui se trouve à la fin des caractères; voir, par exemple, la taille du tableau de messages dans l'exemple précédent et le nombre de caractères qui y sont stockés.

Accès à la matrice

L'indexation des éléments du tableau commence par la valeur 0; l'indice de l'élément 2 dans la matrice mySensVals - dans l'exemple précédent - est 0, l'indice de l'élément 4 (deuxième élément) est 1, et ainsi de suite. Cela signifie également que l'indice du dernier

élément d'un tableau à 10 éléments est 9, et non 10! Voici un exemple:

```
int myArray [10] = {9,3,2,4,3,2,7,8,9,11};
    // La valeur 11 contient l' élément myArray [9]
    // Il fonctionne en dehors du tableau et a myArray [10]
    // Renvoie une valeur aléatoire de la mémoire
```

Pour cette raison, vous devez faire attention à ne pas faire cette erreur lors de l'accès à un élément d'un tableau. L'accès à un élément qui tombe après le dernier élément (indexé supérieur à la taille du tableau avec une valeur de 1), ou généralement en dehors du tableau, conduit à la lecture d'une partie de la mémoire qui est destinée à d'autres fins. La lecture de cette partie ne retourne pas de valeurs valides ou des données. Inversement, écrire sur un morceau de mémoire en dehors de la partie réservée d'un tableau, lors de l'attribution d'une valeur à l'un de ses éléments, est mauvais et plus dangereux que la lecture de cette partie, car cela peut conduire à des résultats désastreux et inattendus tels que des plantages de programmes et temps d'arrêt. Cette erreur peut être difficile à suivre et à affiner.

La dépense de langage C ne vérifie pas l'accès aux tableaux définis dans le programme, bien qu'un accès dépasse les limites d'un, contrairement à BASIC ou JAVA, pour le vérifier.

Attribuer une valeur à un élément du tableau

Toute valeur peut être affectée à un élément du tableau en utilisant l'opérateur d'attribution = et l'index de l'élément en question

Attribuez-lui la valeur comme suit:

mySensVals [0] = 10;

Apportez de la valeur à la matrice

La valeur de tout élément de tableau peut être récupérée par l'index de cet élément comme suit:

x = mySensVals [4];

Tableaux et boucles

Les tableaux sont fréquemment utilisés dans des boucles à modifier ou à chercher les valeurs de leurs éléments. Le compteur de boucles est utilisé comme index de chaque élément du tableau. Par exemple, pour imprimer tous les éléments du tableau sur une interface série, vous pouvez utiliser quelque chose de similaire à l'exemple suivant:

```
int i;
for (i = 0; i <5, i = i + 1) {
  Serial.println (myPins [i]);
}
```

Tapez bool dans Arduino

La variable booléenne déclarée stocke l'une des deux valeurs logiques suivantes: true ou false, en réservant un seul octet de mémoire.

Structure générale

booléen var = val ;

Var représente le nom de la variable à déclarer et val est la valeur à attribuer à cette variable, qui est soit vraie, soit fausse.

Exemples

L'exemple suivant illustre comment utiliser le type booléen pour faire clignoter la lumière attachée à la jambe 5 à chaque pression sur le couteau connecté à la jambe 13:

int LEDpin = 5; // LED light connected to leg 5

int switchPin = 13; // An instantaneous cutter connected to the 13th leg, and the other leg connected to the ground

```
bool running = false;

void setup ()
{
  pinMode (LEDpin, OUTPUT);
  pinMode (switchPin, INPUT);
  digitalWrite (switchPin, HIGH); // Activating the internal lifting resistance
}

void loop ()
{
  if (digitalRead (switchPin) == LOW)
```

```
  {// Was the cutter pressed? Patches resistance keeps voltage always high
    delay (100); // Delay in order to neglect the cutter bounce
    running =! running; // Switch the value of the variable
    digitalWrite (LEDpin, running); // Indicating the state of the variable
through light
  }
}
```

octet dans Arduino

Byte stocke un nombre de 8 bits non signé, c'est-à-dire de 0 à 255.

Structure générale

byte var_name;

byte var_name = Number; // 0 - 255

Exemples

Exemple de définition d'une variable d'octet:

byte myNumber = 125;

byte result = myNuber / 5;

Notez que les nombres après la virgule dans le second cas seront négligés si la division (ou tout calcul) aboutit à une décimale.

Tapez char dans Arduino

Le type char réserve un octet de mémoire et stocke une valeur de police. Les caractères sont écrits entre guillemets simples comme

«A» (pour plusieurs caractères [[[Arduino / chaîne | chaîne de texte]]], utilisez un guillemet pair comme «ABC»).

Les nombres sont stockés dans les octets réservés au type char représentant la valeur du caractère correspondant. Ces nombres sont appelés codage, et le codage utilisé ici est le codage ASCII. Accédez à la documentation de la documentation ASCII pour voir tous les caractères et la valeur numérique correspondante pour chacun. Cela signifie qu'il est possible d'effectuer certains calculs sur les caractères dans lesquels la valeur ASCII est utilisée (comme effectuer l'opération A '+ 1' qui donnera la valeur 66 correspondant à la lettre 'B', puisque la valeur numérique de la lettre «A» dans le symbole ASCII est 65). Jetez un œil à la documentation de la fonction Serial.println () pour plus d'informations sur la façon de changer les caractères et leur valeur numérique.

Char est un type avec un signal; cela signifie que les valeurs numériques qu'elle peut prendre est comprise entre 128 et 127. Si vous voulez un octet sans signal pour stocker des données qu'il contient , utilisez l'octet de type.

Structure générale

cahr var = 'caractère';

char var = nombre;

Var représente le nom de la variable à déclarer et le caractère représente le caractère à attribuer à cette variable. Le numéro de caractère correspondant peut être utilisé en codage ASCII au lieu du même caractère.

Exemples

Exemple de définition d'un type char:

char myChar = 'A';

char myChar = 65; // Les deux représentent la même police

Tapez double dans Arduino

Double est un nombre décimal à double précision. Sur Arduino et autres cartes basées sur ATMEGA, ce type ne réserve que 4 octets (32 bits) de mémoire. Cela signifie que le type double de ces

panneaux présente le comportement du type flotteur lui-même sans aucune différence et sans augmentation de la précision.

Dans les panneaux Due, la taille du type double est de 8 octets (64 bits), soit le double du flottant.

Structure générale

double var;

double var = val ;

Var représente le nom de la variable à déclarer et val représente la valeur décimale à affecter à cette variable.

Exemples

Un exemple de définition d'une variable double:

```
double myNumber = 12.123456789;
double result = myNumber / 2;
```

Notes et avertissements

Les utilisateurs qui apportent des codes contenant des variables de type double provenant d'autres sources doivent essayer de tester le code pour s'assurer que la précision souhaitée ne diffère pas en raison des différentes cartes qui y seront implémentées (en particulier les cartes qui reposent sur des contrôleurs ATmega).

Tapez float dans Arduino

Le type float est utilisé avec des nombres décimaux, qui sont des nombres fractionnaires utilisés comme séparateur décimal. Les nombres décimaux sont souvent utilisés pour se rapprocher des valeurs continues similaires parce qu'ils ont une plus grande précision que les entiers. Les nombres décimaux peuvent être grands avec le nombre 38.028235E + 38 et petits avec le nombre

38-4282835E +. Le type float réserve quatre octets (32 bits) de mémoire pour stocker un float.

Les nombres décimaux flottants ont une petite précision de 6 à 7 décimales après la virgule. Dans Arduino Uno et d'autres cartes de contrôle ATMEGA, il n'y a pas de résolution plus élevée que celle-ci en utilisant le double (double float), car la taille de ce type est exactement la même que celle du float. Dans les panneaux Due, le type double offre une double précision.

Faire des opérations mathématiques sur des nombres décimaux est beaucoup plus lent que d'effectuer les mêmes opérations sur des nombres entiers, donc les nombres décimaux doivent être évités si le timing est critique pour certaines fonctions et que la fonction itérative est nécessaire pour répéter les opérations à la vitesse la plus rapide et dans le temps le plus court possible . Les programmeurs évitent généralement d'effectuer des opérations arithmétiques sur des nombres décimaux et convertissent et manipulent ces opérations pour les effectuer sur des entiers uniquement afin d'augmenter la vitesse du programme.

Si votre programme effectue des opérations mathématiques sur les nombres décimaux, alors vous devez ajouter à tous les nombres fixes utilisés dans ces opérations des décimales après la virgule s'ils ne les ont pas (tels que 12,0); si vous ne le faites pas (c'est-à-dire que vous n'utilisez que le nombre 12), ces nombres seront traités comme des nombres entiers. Voir la page des nombres fixes de la virgule décimale pour plus d'informations.

Structure générale

float var;

float var = val ;

Var représente le nom de la variable à déclarer et val représente la valeur décimale à affecter à cette variable.

Exemples

Un exemple de définition d'une variable flottante et de réalisation d'un calcul dessus:

float myfloat ;

float sensorCalbrate = 1.117;

int x;

int y;

float z;

x = 1;

y = x / 2; // la valeur 0, car elle ne peut pas stocker les parties décimales y contiendra la variable

z = (flottant) x / 2,0; // (la valeur 0.5) faites attention au nombre 2.0 et n'utilisez pas 2 seulement z contiendra la variable

Tapez int dans Arduino

Les nombres entiers sont le type de données principal utilisé pour stocker les nombres.

Sur les cartes Arduino Uno (et les cartes basées sur ATMega), le type int réserve une taille de mémoire de 16 bits (2 octets) dans laquelle des nombres entre 32768- (la plus petite valeur égale à 15 ^ 2) peuvent être stockés et 32767 (le plus grande et égale valeur 1 - 15 ^ 2). Dans les cartes Arduino et les cartes SAMD (telles que MKR1000 et Zero), le type réserve une quantité de mémoire de 32 bits (4 octets) pour stocker des nombres allant de 2147483648- (31 ^ 2-) à 2147483647 (1-31) ^ 2).

Ce type stocke les nombres négatifs avec une technique appelée "Complément à deux". Le bit supérieur le plus important (MSB) détermine si un nombre est négatif ou positif, et il est souvent appelé «bit de signe». Lorsque sa valeur est 1, la valeur de tous les bits restants est inversée et ajoutée à 1.

Arduino a pour tâche de traiter les nombres négatifs, de sorte que les calculs sont effectués sur eux de manière transparente et de la manière attendue. Cependant, il peut y avoir une certaine complexité dans le traitement du coefficient de déplacement <<.

Structure générale

int var = val ;

Var représente le nom de la variable à déclarer et val représente la valeur numérique correcte à affecter à cette variable.

Exemples

L'exemple suivant crée une variable appelée countUp et lui attribue une valeur initiale; cette variable sera utilisée comme compteur croissant, car sa valeur sera augmentée pendant la boucle et apparaîtra sur l'interface de communication série:

int countUp = 0; // Déclarer une variable avec des entiers

```
void setup () {
  Serial.begin (9600); // Configure and start the serial port
}

void loop () {
  countUp ++; // countUp Add the value 1 to the variable
  Serial.println (countUp); // Print the value of the current variable on the serial port
  delay (1000);
}
```

Notes et avertissements

Si les variables de signal dépassent la valeur maximale ou minimale du type dans lequel elles sont définies, elles auront alors atteint un état de "débordement" et leur valeur ne pourra pas être prédite à ce moment. Par conséquent, il faut veiller à ce qu'une variable atteigne l'état d'éruption cutanée (sauf si cela est souhaitable). L'un des indicateurs de l'arrivée de la variable à l'état d'éruption cutanée est le transfert de la variable de sa valeur maximale à sa valeur minimale ou vice versa. Notez que nous avons dit «l'un des indicateurs» et non la seule preuve d'une éruption cutanée, alors soyez prudent. Si

vous définissez une variable et n'avez pas l'intention de l'utiliser pour y stocker une valeur négative, alors le unsigned int de ce type, qui est un unsigned int, doit être utilisé pour réduire la possibilité d'atteindre le débordement.

Cordes dans Arduino

Les chaînes peuvent être représentées de deux manières. Soit les chaînes de texte sont utilisées comme l'un des types de données dans Arduino qui est devenu partie du noyau à partir de la version 0019, soit une chaîne de texte peut être créée à partir d'un tableau de caractères avec le caractère nul à la fin; la deuxième méthode sera expliquée sur cette page, comme la première méthode est expliquée en détail dans la documentation. L'objet string fournit de nombreuses fonctions et outils pour manipuler des chaînes de texte mais au détriment de la mémoire.

Structure générale

Tous les exemples suivants sont des chaînes valides pour les déclarations:

char Str1 [15];

char Str2 [8] = {'a', 'r', 'd', 'u', ' i ', 'n', 'o'};

char Str3 [8] = {'a', 'r', 'd', 'u', ' i ', 'n', 'o', '\ 0'};

char Str4 [] = " arduino ";

char Str5 [8] = " arduino ";

char Str6 [15] = " arduino ";

Il existe plusieurs possibilités pour déclarer des chaînes:

Déclarer une matrice sans formatage comme dans Matrix Str1.

En déclarant une matrice de caractères (en augmentant la taille pour accueillir un caractère supplémentaire), le compilateur ajoutera automatiquement le null requis comme Str2.

Ce cas est comme le cas précédent, sauf pour l'ajout explicite de null au tableau comme Str3.

Configurez le tableau avec une chaîne de texte fixe indiquée entre guillemets; lors du décodage du code, le tableau choisira une taille appropriée pour le tableau qui correspond à la chaîne de texte fixe avec un terminateur nul. Regardez la matrice Str4 pour ce cas.

Initialise le tableau et le spécifie explicitement, comme Str5.

Configurez la matrice avec une option surdimensionnée pour fournir une capacité de chaîne de texte future; voir Str6 par exemple.

Caractère de terminaison nul

En général, les chaînes se terminent par null (ASCII 0). Cela permet aux fonctions (telles que la fonction Serial.print ()) de connaître la fin d'une chaîne de texte lorsqu'elle est lue ou extraite de la mémoire. Si le caractère de fin n'est pas présent, la fonction terminera la lecture ou la récupération d'octets successifs en mémoire qui peuvent ne pas faire partie de la chaîne de texte elle-même.

Cela signifie que toute chaîne de texte doit disposer d'un espace supplémentaire pour un caractère supplémentaire en plus du contenu que vous souhaitez stocker dans la chaîne. C'est la raison pour laquelle Str2 et Str5 ont besoin d'une taille de 8, même si l'expression « arduino » n'est que de 7, car à la dernière position disponible, la police sera automatiquement ajoutée nulle. Pour Str4, sa taille sera automatiquement fixée à 8 caractères, en tenant compte du caractère de fin. Dans la matrice Str3, nous avons ajouté le null null de l'expression ('0 \') manuellement.

Notez qu'il est possible qu'il existe une chaîne de texte qui n'a pas de caractère de fin (tel que la définition de Str2 sur 7 au lieu de 8). Cela causera un dysfonctionnement dans la plupart des fonctions traitant des chaînes, vous devez donc éviter de le faire par inadvertance. Si vous remarquez que le comportement du programme est devenu quelque peu étrange (en particulier lorsqu'il

s'agit de caractères et non de chaînes), alors le caractère de fin est probablement la cause du problème.

Guillemets simples et doubles

Les chaînes sont toujours définies entre guillemets doubles comme "ABC" et les caractères sont toujours définis entre guillemets simples comme "A".

Emballage de longues cordes

Les longues chaînes peuvent être enveloppées et déplacées vers la ligne suivante comme suit:

char myString [] = "Ceci est la première ligne"

"Et cette deuxième ligne"

" Etc";

Matrices de chaînes de texte

Il est souvent approprié lorsqu'il s'agit de traiter une grande quantité de texte, comme travailler sur un projet avec un écran LCD, créer un tableau de chaînes de texte. Puisque les chaînes sont elles-mêmes des matrices, dans ce cas, nous utiliserons une matrice bidimensionnelle.

Dans l'exemple suivant, la présence du paramètre * après avoir défini le type de données char comme "* char" indique que la matrice à déclarer est un tableau de pointeurs. En réalité, tous les tableaux sont des pointeurs, il est donc important d'utiliser ce paramètre * de cette manière pour créer un tableau de tableaux. Les indicateurs en C sont un peu compliqués et pas compris par de nombreux programmeurs novices, mais il suffit de ne comprendre les indicateurs que superficiellement pour les traiter dans notre cas.

Exemples

Un exemple de définition d'un tableau à l'aide d'un paramètre * et d'impression du contenu de ce tableau sur l'interface série:

```
char * myStrings [] = {"This is string 1", "This is string 2", "This is string 3",
"This is string 4", "This is string 5", "This is string 6"};

void setup () {
Serial.begin (9600);
}

void loop () {
for (int i = 0; i <6; i ++) {
  Serial.println (myStrings [i]);
  delay (500);
  }
}
```

Tapez un mot dans Arduino

Les variables de type mot stockent un nombre non-signal de 16 bits (2 octets), avec une valeur comprise entre 0 et 65 535 max. Ce type est similaire au type int non signé.

Structure générale

word var = val;

Var représente le nom de la variable à déclarer et val représente la valeur à attribuer à cette variable.

Exemples

Exemple de définition d'une variable de type mot et de lui affecter une valeur:

word w = 10000;

Le mot-clé void dans Arduino

Le mot clé void est utilisé avec les fonctions définies par le programmeur dans le code pour indiquer que cette fonction ne devrait pas renvoyer de données à celui qui l'a appelée après son exécution. Sur la base de cette définition, le mot-clé return n'est pas supposé être utilisé dans la fonction déclarée avec le mot-clé void.

Structure générale

```
void function ()
{
   // Le bloc de code que le code exécutera
```

}

Exemples

Le code suivant explique comment utiliser void avec la fonction setup () et loop () pour indiquer qu'ils ne retourneront rien une fois leur implémentation terminée:

```
// Mais "loop" et "setup" ne seront pas exécutés pour toutes les tâches des deux fonctions
// Aucun d'eux ne renvoie de données au programme supérieur qui les a appelés

void setup ()
{
  // ...
}

void loop ()
{
  // ...
}
```

Chapitre III

Constantes et variables

Entiers fixes dans Arduino

Les entiers fixes sont des nombres utilisés directement dans le code comme 123. Par défaut, ces nombres sont traités comme des entiers mais peuvent être convertis en d'autres types de nombres à l'aide d'un modificateur tel que U ou L.

Les entiers sont traités comme des entiers basés sur le nombre 10 toujours (c'est-à-dire représentés par décimal par défaut). Cependant, d'autres formules peuvent être utilisées pour écrire des nombres, car la base du nombre varie.

Système décimal (base 10)
Ce système est le système de comptage commun chez les humains qu'ils utilisent toujours dans les calculs. Les nombres écrits dans le code qui ne sont précédés d'aucun coordinateur (préfixe déterminant la formule numérique) sont considérés comme des nombres écrits en décimal (base 10).

n = 101; // ((1 * 10 ^ 2) + (0 * 10 ^ 1) + 1) == 101

Système binaire (base 2)

Les valeurs de ce système sont représentés par des nombres 0 et 1 uniquement.

n = B101; // ((1 * 2 ^ 2) + (0 * 2 ^ 1) + 1 * 2 ^ 0) == 5 en décimal

La formule binaire n'est valide que pour les valeurs d'octets (8 bits); entre 0 (B0) et 255 (B11111111). Si vous souhaitez utiliser la formule binaire avec des valeurs plus élevées - comme des entiers (16 bits) - vous pouvez le faire en effectuant un processus simple en deux étapes comme suit:

myInt = (B11001100 * 256) + B10101010; // est la valeur d'octet supérieur B11001100

Huit (base 8)

Les chiffres utilisés pour représenter les valeurs de ce système sont de 0 à 7. Les valeurs à écrire et représentées dans ce système commence par le coordinateur 0 (zéro).

n = 0101; // ((1 * 8 ^ 2) + (0 * 8 ^ 1) + 1) == 65 en décimal

Une erreur logique difficile à trouver dans le code est susceptible de résulter d'une constante zéro non intentionnelle; cela conduit l'interprète à interpréter cette constante comme une valeur de huit pendant la décharge du code.

Système hexadécimal (base 16)

Les chiffres et les caractères utilisés pour représenter les valeurs de ce système sont de 0 à 9 et A à F. En fait, la valeur décimale de la lettre A est 10, la lettre B est 11, et ainsi de suite jusqu'à ce que la lettre F est 15. les valeurs à être représentées et écrites dans le système hexadécimal avec 0x préfixe. Notez que les lettres AF peuvent être utilisées en majuscules ou minuscules (af).

n = 0x101; // ((1 * 16 ^ 2) + (0 * 16 ^ 1) + 1) == 257 en décimal

Notes et avertissements

Les commutateurs U et L

Par défaut, un entier constant est traité comme un type d'entier dont la taille est limitée et insuffisante dans certains cas. Pour changer ce type en un autre type de données, vous pouvez utiliser l'un des commutateurs U ou L comme suit:

'u' ou 'U': ce commutateur est utilisé à la fin de la constante numérique pour la convertir en un type non signé tel que 33u.

«l» ou «L»: Ce commutateur est utilisé à la fin de la constante numérique pour la convertir en un type «nombre long» tel que 100000L.

«ul» ou «UL»: les deux commutateurs sont utilisés ensemble à la fin de la constante numérique pour la convertir en un type «nombre long sans signal» tel que 32767ul.

Constantes dans Arduino

Les constantes sont des expressions prédéfinies dans le langage Arduino. Utilisé pour faciliter le travail des programmeurs lors de l'écriture du code, ainsi que pour rendre le code plus lisible. Les constantes sont regroupées par fonction.

Définition des niveaux logiques (constantes logiques)

Dans Arduino, deux constantes sont utilisées pour représenter la vérité et la fausseté: vraie et fausse.

La constante est fausse

La valeur fausse logique est une valeur d'erreur et est définie numériquement comme zéro (0).

La constante est vraie

Un booléen vrai est une valeur vraie et est défini comme un (1) numériquement. En fait, le vrai représente un plus large éventail de valeurs numériques que juste 1. Toute valeur non nulle est

logiquement appelé vrai. Par conséquent, les valeurs 1, 2, et 200 tout logiquement indiquer le vrai.

Sachez que vrai et faux sont en minuscules, contrairement à d'autres constantes telles que HIGH, LOW, INPUT, etc.

Définition de l'état des jambes (niveau de tension)

Lors de la lecture ou l' écriture de la valeur des jambes numériques, nous constatons que nous avons affaire à deux valeurs qui ne sont pas tiers; l'une des jambes numériques peut être prise ou réglée sur HIGH ou LOW.

Hard HIGH

La signification de la valeur HIGH fixe, lorsqu'il s'agit des jambes de la carte Arduino, varie en fonction de la position de l'homme; c'est-à-dire que l'homme est une INPUT ou une OUTPUT. Si l'homme est défini sur un homme qui est entré en passant la valeur INPUT à la fonction pinMode () et est lu à l'aide de la fonction digitalRead (), le contrôleur ATmega signalera la valeur HIGH si:

La tension appliquée à la jambe est supérieure à 3,0 V sur les panneaux avec une tension de 5 V, ou

La tension appliquée à la jambe est supérieure à 2,0 V dans les panneaux 3,3 V.

L'homme peut également être défini sur un homme d'entrée en passant la constante INPUT à la fonction pinMode (), puis en tapant la valeur HIGH dessus en utilisant la fonction digitalWrite (). Cela activera la résistance de levage interne associée à cet homme. L'activation de cette résistance augmentera l'effort de l'homme à HIGH à moins qu'il ne soit réduit à LOW en le connectant à un circuit externe. Ceci est très similaire à ce que fait la constante

INPUT_PULLUP lorsqu'elle est passée à la fonction pinMode (), que nous aborderons plus tard.

Quand un homme est défini sur un homme de sortie en passant la constante OUTPUT à la fonction pinMode (), puis en tapant la valeur HIGH dessus en utilisant la fonction digitalWrite (), l'effort de cet homme sera:

5 volts en 5V, ou

3.3V dans les panneaux 3.3V.

Lorsqu'un homme est dans cet état, alors un courant peut être tiré pour allumer une lumière LED, par exemple en connectant l'un de ses pôles à celui-ci et en connectant l'autre pôle avec une résistance sur la séquence au pôle de masse.

Dur FAIBLE

La signification de la valeur LOW fixe varie également en fonction de la position de l'homme; Si l'homme est défini sur un homme d'entrée en passant la valeur INPUT à la fonction pinMode () et est lu à l'aide de la fonction digitalRead (), le contrôleur ATmega rapportera la valeur LOW si:

La tension appliquée à la jambe est inférieure à 1,5 V dans les panneaux avec une tension de 5 V, ou

La tension appliquée à la jambe est inférieure à environ 1,0 V dans les panneaux de 3,3 V.

Quand la position d'un homme est définie sur un homme de sortie en passant la constante OUTPUT à la fonction pinMode (), puis en tapant la valeur LOW dessus à l'aide de la fonction digitalWrite (), la tension de cet homme est de 0 volts (à la fois en 5V et 3,3V panneaux). Lorsque l'homme est dans cette situation, un courant

peut être déchargé, tel qu'un pôle lumineux qui lui est connecté et l'autre pôle avec résistance sur la séquence avec un pôle 5V + (ou 3,3V +) pour l'allumer.

Définition de la position des jambes (entrée ou sortie)

Les jambes numériques peuvent être utilisées comme jambes d'entrée ou comme jambes de sortie en utilisant l'une des constantes INPUT, INPUT_PULLUP ou OUTPUT avec la fonction pinMode (). Quand un homme est ainsi préparé, son comportement électrique est complètement changé en interne.

ENTRÉE dure

Les jambes de la carte Arduino définies comme jambes d'entrée à l'aide de la constante INPUT avec la fonction pinMode () sont dites à l'état haute impédance. Les jambes fixées aux jambes INPUT rendent les charges sur le circuit que vous examinez très petites, équivalentes à une résistance de séquence de 100 ohms devant l'homme. Cela rend les jambes très utiles pour lire les valeurs des capteurs.

Si l'une des jambes est définie comme un homme d'entrée et utilisée pour lire un interrupteur, la condition de l'homme sera «flottante» lorsque le couteau est ouvert; cela produira des résultats inattendus. Pour vous assurer que l'état de la jambe est lu correctement lorsque le couteau est ouvert, utilisez une résistance externe de traction vers le haut ou vers le bas avec cette jambe. Le but de la connexion de la résistance à cet homme est d'ajuster son effort à un état connu lorsque le couteau est ouvert. Une résistance de 10 k أوم est généralement utilisé, ce qui est suffisamment petit pour empêcher la jambe d'entrée de flotter, et assez grand pour ne pas tirer une grande quantité de courant lorsque le couteau est fermé. Consultez ce didacticiel pour plus d'informations.

Si une résistance de tirage vers le bas externe est utilisée, la tension de la jambe d'entrée sera basse lorsque le couteau est ouvert et HIGH lorsque le couteau est fermé.

Si une résistance de tirage vers le haut externe est utilisée, la tension de la jambe d'entrée sera ÉLEVÉE lorsque le couteau est ouvert et BAS lorsque le couteau est fermé.

La constante INPUT_PULLUP

Le contrôleur ATmega de la carte Arduino a une résistance de levage interne connectée à chaque jambe des jambes d'entrée et de sortie qui peuvent être contrôlées. Si vous préférez utiliser cette résistance au lieu d'ajouter une résistance de levage externe, vous pouvez le faire en passant la constante INPUT_PULLUP à la fonction pinMode () lors de la configuration de la jambe. Consultez ce tutoriel, qui contient un exemple pratique de ce que nous avons mentionné.

AVERTISSEMENT: les pieds réglés comme pieds d'entrée à l'aide de INPUT ou INPUT_PULLUP peuvent être endommagés ou détruits s'ils sont connectés à une tension inférieure à la tension de terre (tension négative) ou supérieure à la tension positive (5 V ou 3,3 V).

SORTIE dure

On dit que les jambes de la carte Arduino définies comme les jambes de sortie en utilisant la constante OUTPUT avec la fonction pinMode () sont dans un état de haute impédance. Cela signifie qu'ils peuvent fournir suffisamment de courant aux autres appareils et circuits qui leur sont connectés. Tout homme avec des jambes ATmega peut donner ou décharger d'autres appareils et circuits

jusqu'à 40 mA. Le dépassement de cette valeur de courant endommage le contrôleur, c'est pourquoi des résistances ou des éléments électroniques similaires doivent être utilisés pour déterminer le courant lors de la connexion de pièces électroniques qui nécessitent de grandes quantités de courant telles que des moteurs.

AVERTISSEMENT: les pattes définies comme pattes de sortie à l'aide de OUTPUT peuvent être endommagées ou vandalisées si elles sont connectées à la tension de terre ou à la tension positive de la source de courant directement utilisée.

Définir des éléments incorporés dans le panneau

LED_BUILTIN fixe

La plupart des cartes Arduino ont une lumière LED intégrée dans la carte et connectée à une jambe avec une résistance sur la séquence. La constante LED_BUILTIN représente le numéro de l'homme connecté à ce panneau lumineux. Cet homme est le 13e homme dans la plupart des peintures Arduino qui ont une lumière intégrée.

Portée variable et qualificatifs

Le mot-clé const dans Arduino

Le mot-clé const est une abréviation du mot constant, qui est contraint par les variables qui sont utilisées avec lui. Cela signifie qu'une variable peut être utilisée comme représentative de variables du même type, à l'exception de l'impossibilité de modifier la valeur

qui lui a été affectée lors de sa création. Vous rencontrerez une erreur du banquier si vous essayez d'attribuer une valeur à une variable avec laquelle le mot-clé const est utilisé.

Les constantes définies à l'aide du mot clé const sont conformes aux règles de portée des variables qui définissent le reste des variables. En tenant compte des risques de define #, cela fait du mot-clé const le meilleur moyen de définir des constantes, et de préférence les utiliser au lieu de define #.

Structure générale

const dataType var = val ;

Le dataType représente le type de variable à définir, var le nom de la variable et val la valeur à affecter à cette variable constante. Notez qu'une fois que val est affecté à une variable, il ne peut plus être modifié et la variable sera en lecture seule.

Exemples

Utilisez le mot const pour définir la constante π utilisée dans les opérations mathématiques:

```
const float pi = 3.14;
float x;

// ....

x = pi * 2; // Constants are frequently used in mathematical operations
```

pi = 7; // This will generate an error, as a constant variable cannot be overwritten

Notes et avertissements

Expression définir # ou mot-clé const?

Vous pouvez utiliser le mot-clé const ou #define pour créer des constantes numériques ou textuelles. Pour les tableaux, il vous suffit d'utiliser const avec eux. En général, l'utilisation du mot-clé const est préférable à l'utilisation de l'expression define #.

Le domaine des variables dans Arduino

Les variables du langage C, dont Arduino est dérivé, ont une propriété appelée scope qui détermine la portée d'utilisation de ces variables et qui peut y accéder . Ceci est différent des versions récentes des langues telles que BASIC où la variable est toujours globale.

Les variables générales peuvent être vues par toutes les fonctions du programme. En revanche, les variables locales ne sont visibles que dans la fonction dans laquelle elles sont définies et ne sont pas disponibles pour d'autres fonctions. Dans l'environnement IDE, toute variable déclarée en dehors d'une fonction (telle que setup (), loop (), etc.) est une variable globale.

Lorsqu'un programme devient plus volumineux et plus complexe, les variables locales sont plus utiles que les variables générales pour garantir qu'une seule fonction atteigne les variables qui y sont

définies. Cela empêche les erreurs d'une fonction de modifier des variables qu'une autre fonction utilise par inadvertance.

Il est utile de déclarer des variables et de leur attribuer une valeur dans des boucles itératives telles que for. Cela empêche tout ce qui se trouve en dehors de la boucle d'accéder et de modifier cette variable.

Exemples

```
int gPWMval; // Any function can see and access this variable

void setup ()
{
  // ...
}

void loop ()
{
  int i; // loop is only visible within the variable "i" loop
  float f; // Only the loop is also visible within the variable "f" loop
  // ...

  for (int j = 0; j <100; j ++) {
  // only for available within the variable j
  }

}
```

Le mot-clé statique dans Arduino

Le mot-clé static est utilisé lors de la création de variables visuelles pour une seule fonction afin de préserver son contenu après l'exécution de cette fonction invoquée et jusqu'à son prochain appel, contrairement aux variables locales qui sont créées et détruites chaque fois que la fonction est appelée.

Les variables statiques déclarées dans la fonction ne seront créées et initialisées avec le mot-clé static que la première fois que la fonction est appelée.

Structure générale

static dataType var = val;

Le dataType représente le type de variable à définir, var le nom de la variable et val la valeur à affecter à cette variable statique. Sachez qu'une fois que la variable var est créée lors du premier appel de la fonction, elle ne sera pas détruite lors de l'exécution de la fonction et la valeur qui y est stockée sera conservée jusqu'au prochain appel de la fonction.

Exemples

Le programme suivant génère des étapes aléatoires limitées à un domaine spécifique et limite également la taille des étapes via la variable stepsize passée à la fonction randomWalk ():

/ * RandomWalk

* Paul Badger 2007

* RandomWalk wanders up and down randomly between two

* Endpoints. The maximum move in one loop is governed by

* the parameter "stepsize".

* A static variable is moved up and down a random amount.

* This technique is also known as "pink noise" and "drunken walk".

```
#define randomWalkLowRange -20
#define randomWalkHighRange 20
int stepsize;

int thisTime;
int total;
```

```
void setup ()
{
  Serial.begin (9600);
}

void loop ()
{// randomWalk test function
  stepsize = 5;
  thisTime = randomWalk (stepsize);
  Serial.println (thisTime);
   delay (10);
}

int randomWalk (int moveSize) {
  static int place; // is defined as static so it stores values
randomWalk is a variable that stores a value in the function
                    // Between function calls with this variable
remaining local and only accessed by this function


  place = place + (random (-moveSize, moveSize + 1));


  if (place <randomWalkLowRange) {// Check the upper and
lower limits
    place = randomWalkLowRange + (randomWalkLowRange -
place); // Reverses the number in the opposite direction to the
positive direction
  }
  else if (place> randomWalkHighRange) {
```

```
    place = randomWalkHighRange - (place -
randomWalkHighRange); // Reverses the number in the
opposite direction to the negative direction
  }

  return place;
}
```

Mot-clé volatile dans Arduino

Le mot-clé volatile est classé sous «qualificatif de variable», et est généralement utilisé avant le type de variable lorsqu'il est défini pour modifier la façon dont le banquier et le programme suivant traitent cette variable.

Le mot clé volatile, lorsqu'il est utilisé pour définir une variable, est une directive pour le banquier lui-même. Le banquier est un programme qui convertit le code C / C ++ en un code exécutable que le matériel comprend; ce code contient des commandes réelles pour le contrôleur sur la carte Arduino lui disant quoi faire.

Ce que le mot-clé volatile fait avec la variable est de diriger le banquier pour charger la variable à partir de la RAM et non à partir des enregistrements de la mémoire flash où toutes les variables du programme sont stockées. Dans certains cas, la valeur stockée dans les enregistreurs pour une variable peut être inexacte.

La variable doit être définie avec le mot-clé volatile chaque fois que sa valeur peut être modifiée par quoi que ce soit hors du contrôle du

fragment de code dans lequel cette variable apparaît (comme l'exécution simultanée d'une chaîne). Dans Arduino, le seul endroit où cela est susceptible de se produire est les fragments de code liés au district appelés programmes ISR.

Entiers volatils ou longs?

Si vous utilisez le mot volatile avec une variable supérieure à un octet (2 octets en entier, 4 octets en nombre long, etc.), le contrôleur ne pourra pas le lire en une seule étape car l'architecture du contrôleur est souvent 8 bits (assurez-vous que l'architecture est le panneau de contrôle). Cela signifie que tandis que les huit premiers bits de la variable sont lus dans une partie du code (comme la boucle), l'interruption peut avoir déjà changé les huit autres bits (si la taille de la variable est de 16 bits). Cela obtiendra une valeur aléatoire pour la variable.

La solution à ce problème est de désactiver les interruptions pendant la lecture des valeurs variables, de sorte que les valeurs des bits variables ne changent pas comme ils sont lus. Il y a plusieurs moyens de le faire:

Utilisez la fonction noInterrupts () pour désactiver les interruptions, ou

L'utilisation du bloc ATOMIC_BLOCK, où les opérations atomiques sont des opérations uniques effectuées par le contrôleur (MCU) et représentent la plus petite unité possible.

Exemples

Changez l'état du pied connecté avec une lumière LED lorsqu'une interruption se produit sur le pied 2:

```
int pin = 13;
volatile byte state = LOW;

void setup ()
{
  pinMode (pin, OUTPUT);
  attachInterrupt (digitalPinToInterrupt (2), blink, CHANGE);
}

void loop ()
{
  digitalWrite (pin, state);
}

void blink ()
{
  state =! state;
}
```

Chapitre IV

Structures de contrôle

Rupture de mot-clé dans Arduino

Le mot-clé break est utilisé pour sortir des boucles pendant, while ou do ... pendant l'itération, en contournant le code suivant et les conditions de boucle spécifiées et en les quittant. Également utilisé pour quitter l'expression de cas du commutateur programmatique ...

Structure générale

Pause;

Exemples

Dans le code suivant, le contrôleur quitte la boucle for lorsque la valeur mesurée du capteur dépasse un certain seuil:

```
for (x = 0; x <255; x ++)
{
   analogWrite (PWMpin, x);
   sens = analogRead (sensorPin);
   if (sens> threshold) {// Ensure that the measured value does not exceed the threshold
      x = 0;
      break;
   }
```

```
    delay (50);
}
```

Continuer le mot-clé dans Arduino

La clé continue transmet le code suivant dans la boucle d'itération (comme for, while ou do ... while iterations) pour vérifier l'expression conditionnelle de cette boucle, puis la boucle de boucle reprend normalement. Faire cette instruction est similaire à faire un saut dans la boucle pour sauter l'exécution lorsqu'une certaine condition est remplie.

Structure générale

continuer;

Exemples

Dans le code suivant, la valeur 0 à 255 sera écrit sur le pied PWMpin avec sauter les valeurs dans la gamme 41-119:

```
for (x = 0; x <= 255; x ++)
{
    if (x> 40 && x <120) {// Causes a jump in values
        continue;
    }

    analogWrite (PWMpin, x);
    delay (50);
```

}

Bouclez ... tout en itératif dans Arduino

La boucle itérative do ... while fonctionne de la même manière que la boucle while sauf que la condition est vérifiée à la fin de la boucle, pas au début. Cela signifie que la boucle sera exécutée au moins une fois.

Structure générale

```
do
{
   // The code block to be executed
} while (condition);
```

Une condition est une expression logique et sera évaluée comme vraie ou fausse.

Exemples

Un exemple d'utilisation de la boucle do ... while pour lire une valeur d'un capteur et attendre qu'elle atteigne une certaine limite:

```
do
{
  delay (50); // Wait for the sensor value to stabilize
```

```
  x = readSensors (); // Read the value of the sensor

} while (x <100);
```

Mot-clé else dans Arduino

L'expression conditionnelle if ... else permet plus de contrôle sur la séquence de validation de plusieurs conditions spécifiques lors de l'exécution du code plutôt que de vérifier une seule condition lors de l'utilisation de l'expression if. L'instruction else (le cas échéant) sera exécutée si la condition donnée dans l'instruction if n'est pas remplie. Vous pouvez également utiliser l'instruction else avec l'instruction if comme else if pour ajouter une autre condition à vérifier, afin que plusieurs vérifications consécutives puissent être effectuées en même temps.

S'il y a plusieurs conditions à vérifier, elle sera remplie à partir de la première condition, puis à partir de la condition suivante si elle retourne false et ainsi de suite jusqu'à ce que l'une des conditions soit remplie (ie retourne true). Lorsque la condition est remplie, le bloc associé sera exécuté et le programme sautera toute la structure if / else à la ligne suivante. Si aucune condition n'est remplie, le bloc else par défaut sera exécuté s'il se produit finalement.

Notez que le bloc else if peut être utilisé avec ou sans le bloc else à la fin et vice versa. Le nombre de fois où l'utilisation du bloc else if dans une expression conditionnelle est illimitée.

Structure générale

```
If (Condition 1)
{
  // Do the thing a
}
else if (Condition 2)
{
  // Do the thing b
}
else
{
  // Do the thing c
}
```

Examples

The following code is part of the temperature sensing system code:

```
if (temperature> = 70)
{
  // Danger! Stop the system
}
else if (temperature> = 60 && temperature <70)
{
  // Warning! You must send signals to alert the user
}
else
{
  // Safety! Complete the normal tasks
}
```

Épisode pour itératif dans Arduino

La boucle for est utilisée pour dupliquer un bloc de code spécifique un nombre spécifié de fois. Dans cette boucle, un compteur est généralement utilisé pour suivre le nombre de fois où le code est dupliqué et interrompu lorsqu'un nombre spécifié est atteint. La boucle for peut être utilisée avec n'importe quelle itération et est fréquemment utilisée avec des tableaux pour effectuer une opération particulière sur des éléments qui peuvent être des données normales ou des segments spécifiques.

Structure générale

```
for (int x = 2; x <100; x = x * 1.5) {
println (x);
}
```

L'initialisation n'est effectuée qu'une seule fois au début de l'exécution de la boucle. La condition est vérifiée à chaque fois que la boucle est répétée; si vrai, le bloc qui fait partie de la boucle sera exécuté, puis l'étape sera exécutée et la condition sera revérifiée. Lorsque la condition n'est pas remplie (c'est-à-dire que la valeur devient fausse), la boucle cesse de fonctionner.

Exemples

Le code suivant fait partie du code du système de détection de température:

```
int PWMpin = 10; // Lumière LED connectée à la séquence avec une impédance de 10 ohms de 10 pieds


void setup ()
{
  // Cet exemple ne nécessite aucune configuration
}


boucle vide ()
{
   pour (int i = 0; i <= 255; i ++) {
      analogWrite ( PWMpin , i );
      retard (10);
   }
}
```

Notes et avertissements

La boucle en C est plus souple que pour les boucles dans certaines langues telles que BASIC. Un ou tous les trois éléments principaux peuvent être supprimés de la boucle, en conservant le point-virgule intact. En outre, les expressions d'initialisation, de condition et d'étape peuvent être des expressions de langage C valides avec des variables non liées, et peuvent utiliser n'importe quel type de données en C, y compris les décimales. L'utilisation de ces types d'expressions inhabituels avec la boucle for fournit une solution à certains problèmes logiciels rares.

Par exemple, l'utilisation de la multiplication dans l'étape d'expression générera une progression logarithmique telle que:

pour (int x = 2; x <100; x = x * 1,5) {

println (x);

}

Ce code générera les étapes suivantes:

2,3,4,6,9,13,19,28,42,63,94

Un autre exemple d'allumage et d'éteindre progressivement la lumière LED avec un seul anneau:

```
void loop ()
{
  int x = 1;
  for (int i = 0; i> -1; i = i + x) {
    analogWrite (PWMpin, i);
    if (i == 255) x = -1; // Switch the direction of the step when the peak is reached
    delay (10);
  }
}
```

Aller à mot-clé dans Arduino

Le mot-clé goto déplace le programme vers une position spécifique dans le code.

goto label; // Envoie le programme pour exécuter le bloc de code
// étiquette trouvée sur l'étiquette

Exemples

Example of using goto:

```
for (byte r = 0; r <255; r ++) {
   for (byte g = 255; g> 0, g -) {
      for (byte b = 0; b <255; b ++) {
         if (analogRead (0)> 250) {goto bailout;}
         // Something can be added here
      }
   }
}
```

renflouement:

Notes et avertissements

L' instruction goto n'est plus utilisée en C, et dans certains manuels de programmation C, il est indiqué que l' instruction goto n'est plus nécessaire mais peut être utilisée pour simplifier certains programmes. La raison pour laquelle les programmeurs sont réticents à utiliser l' instruction goto est que lorsqu'ils sont ignorés, un programme avec un chemin d'exécution inconnu sera créé, ce qui rend presque impossible le suivi et la correction des erreurs.

D'un autre côté, il existe des situations où l'utilisation du mot goto est utile et simplifie grandement le code. L'un de ces cas est une fragmentation de boucle de fourche ou un bloc if conditionnel à certaines conditions.

Expression si conditionnelle dans Arduino

L'expression conditionnelle if vérifie une condition donnée et exécute le bloc de code qui y est écrit si elle est vraie (c'est-à-dire que sa valeur est vraie).

Structure générale

```
if (condition)
{
  // The block of code to execute when the condition is met
}
```

Une condition est une expression logique dont la valeur est vraie ou fausse.

Exemples

Un exemple d'utilisation de l'expression conditionnelle if dans plusieurs formats:

```
if (x> 120) digitalWrite (LEDpin, HIGH);
```

```
if (x> 120)
```

```
digitalWrite (LEDpin, HIGH);

if (x> 120) {digitalWrite (LEDpin, HIGH); }

if (x> 120) {
  digitalWrite (LEDpin1, HIGH);
  digitalWrite (LEDpin2, HIGH);
}
```

Notes et avertissements

Les termes donnés peuvent nécessiter l'utilisation de l'un des paramètres suivants.

Opérations budgétaires

x == y (y est égal à x)

X! = y (y n'est pas égal à x)

x <y (y est exactement plus petit que x)

x> y (y est exactement supérieur à x)

x <= y (y est plus petit ou égal à x)

x> = y (y est supérieur ou égal à x)

Veillez à ne pas utiliser un seul signe égal (par exemple si (x = 10)). Égale (comme if (x == 10)), qui dans ce cas est un coefficient d'équilibre et vérifie si la valeur de la variable x est égale ou non. La valeur de la dernière expression sera vraie si la valeur de la variable

x est 10, tandis que la valeur de l'expression de référence précédente est toujours vraie.

L'explication de la façon dont C évalue l'expression si (x = 10) avec vrai est toujours que C attribue la valeur 10 à la variable x (rappelez-vous qu'un seul signe égal est un opérateur d'attribution et a été utilisé par erreur). L'expression conditionnelle if évalue ensuite la valeur 10 - qui devient la valeur de la variable x - comme toujours vraie car l'évaluation logique de tout nombre différent de zéro est toujours vraie. Par conséquent, l'expression conditionnelle if (x = 10) sera évaluée comme vérifiée et le résultat sera contraire aux attentes attendues à ce moment-là. De plus, la variable x sera mise à 10, ce qui n'est pas non plus souhaitable

Retour de mot-clé dans Arduino

Le mot-clé return croise l'exécution de n'importe quelle fonction et renvoie une valeur de la fonction à celui qui l'a appelée si spécifié.

Structure générale

return;

return value; // value In this format, the value will be returned

La valeur renvoyée peut être n'importe quel type de variable ou de constantes à renvoyer à celui qui a appelé la fonction.

Exemples

L'exemple suivant montre comment créer une fonction pour équilibrer la valeur mesurée d'un capteur avec un seuil spécifique:

```
int checkSensor () {
   if (analogRead (0)> 400) {
      return 1;
   }
   else {
      return 0;
   }
}
```

L'exemple suivant montre comment utiliser le mot clé return pour examiner une partie du code sans recourir à la méthode comment:

```
void loop () {

// Cette partie du code est vérifiée

return;

// La partie ici qui ne sera pas implémentée
// L'erreur est probablement dedans
}
```

Boîtier de commutateur d'expression en Arduino

Le cas switch ..., comme si, vérifie une valeur spécifiée d'une manière qui permet au programmeur de spécifier plusieurs codes, dont l'un est exécuté en fonction de cette valeur. De manière plus détaillée, le commutateur ... cas équilibre la valeur d'une variable avec les valeurs spécifiées en son sein; lorsque la valeur de cette variable correspond à l' une des valeurs ou des expressions en son sein, sera exécuté le bloc associé à cette valeur ou une expression.

Le mot clé break est généralement utilisé à la fin de chaque état du switch ... case pour le quitter après avoir mis en correspondance l'état et exécuté le bloc associé. Si le key break n'est pas utilisé dans ce cas, le cas switch ... continuera à examiner les conditions et à exécuter les blocs associés jusqu'à ce que le break soit atteint ou à la fin.

Structure générale

```
switch (var) {
  case label1:
    // The first code that will be executed when the first case is matched
    break;
  case label2:
    // The second code will be executed when the second case is matched
    break;
  default:
    // The default code that will be executed when no case matches
    // This case is optional
}
```

Transactions

var

La variable d'équilibrer sa valeur à valeurs multiples (états). Les types de données autorisés sont: Entier, police.

lable1

La première constante qui sera équilibrée avec var. Les types de données autorisés sont: Entier, police.

lable2

La deuxième constante qui sera équilibrée avec var. Les types de données autorisés sont: Entier, police.

Exemples

Un exemple d'utilisation de l'expression switch ... case pour équilibrer une valeur de variable et exécuter un code le cas échéant:

```
switch (var) {
  case 1:
    // Value 1 var Do something when it equals
    break;
  case 2:
    // Value 2 var Do something when it equals
    break;
  default:
    // with any of the previous values   var do something if it doesn't match
    break;
}
```

Alors que l'épisode itératif dans Arduino

La boucle se répète tout en exécutant le bloc de code en son sein de manière continue et infinie tant que la valeur de la condition logique entre parenthèses est vraie (voir l'évidence générale); une fois que la valeur de la condition logique devient fausse, la boucle s'arrête et se termine. Toute modification de la valeur de la condition doit se produire sinon la boucle ne sortira jamais, donc la valeur de la variable qui est vérifiée dans la condition logique doit être augmentée ou diminuée, ou une condition externe telle que la vérification d'une valeur sensible ou similaire doit être utilisé.

Structure générale

```
while (condition) {
  // bloc à dupliquer
}
```

Transactions

état

Une expression logique dont la valeur est vraie ou fausse.

Exemples

Un exemple d'utilisation de l'expression while pour répéter l'exécution d'un code un certain nombre de fois:

```
var = 0;
while (var <200) {
  // The code written here will be repeated 200 times
  var ++;
}
```

Chapitre V

Les fonctions

Fonctions de commande au pied numérique dans Arduino

La fonction digitalRead () dans l'Arduino

La fonction digitalRead () lit la valeur d'un pied numérique qui est soit HIGH ou LOW.

Structure générale

digitalRead (broche)

Transactions

épingle

Un nombre indiquant l'homme numérique dont la valeur doit être lue.

Valeurs renvoyées

Renvoie la valeur HIGH ou LOW représentant la valeur numérique du pied d'épingle.

Exemples

Réglez la valeur du pied 13 sur la même valeur que celle du pied 7 définie comme pied d'entrée:

```
int ledPin = 13; // There is a LED light connected to the 13 foot digital
int inPin = 7; // Push button connected to 7 digital foot
int val = 0; // Changer to store the read value

void setup ()
{
  pinMode (ledPin, OUTPUT); // 13 digital foot set as the output foot
```

```
  pinMode (inPin, INPUT); // Set the 7 foot digital as the entered foot
}

void loop ()
{
  val = digitalRead (inPin); // Read the income blood
  digitalWrite (ledPin, val); // Adjust the light status to the value of the button
}
```

Notes et avertissements

Si vous ne connectez le pied à rien, la fonction digitalRead () peut renvoyer soit HIGH soit LOW (cela peut changer de manière aléatoire).

Des pieds d'entrée similaires peuvent également être utilisés comme pieds numériques, appelés alors A0, A1, etc.

La fonction digitalWrite () dans l'Arduino

La fonction digitalWrite () écrit la valeur HIGH ou la valeur LOW sur une jambe numérique spécifique.

Si l'homme est configuré en tant que sortie (c'est-à - dire , OUTPUT) via la fonction pinMode (), la tension de cet homme est réglée à la valeur correspondante: 5V (ou 3,3V dans les panneaux 3,3V) pour HIGH, ou 0V (masse) pour LOW .

Si le man est configuré comme INPUT, il activera (avec HIGH) ou désactivera (avec LOW) la fonction digitalWrite (résistance pullup interne) associée à cette entrée man. Il est recommandé d'utiliser la valeur INPUT_PULLUP avec la fonction pinMode () pour activer directement la résistance de levage interne. Voir le "Tutoriel des jambes numériques" pour plus d'informations.

Si vous ne configurez pas un homme pour OUTPUT via la fonction pinMode () et connectez une lumière LED avec elle, la lumière LED peut sembler faible lors de l'appel digitalWrite (HIGH) avec ce gars. Sans configurer explicitement l'homme via la fonction pinMode (), digitalWrite () activera la résistance de levage interne qui régulera le grand courant circulant alors.

Structure générale

digitalWrite (broche, valeur)

Transactions

épingle

Un nombre indiquant l'homme numérique sur lequel écrire la valeur.

valeur

La valeur HIGH ou LOW à écrire sur le pin man.

Valeurs renvoyées

Rien n'est retourné.

Exemples

Réglez le mode numérique 13 homme sur l'homme de sortie, puis inversez sa valeur de HIGH à LOW et vice versa toutes les secondes:

```
void setup ()
{
  pinMode (13, OUTPUT); // 13 digital man set it as a man came out
}

void loop ()
{
  digitalWrite (13, HIGH); // HIGH Adjust the value of 13 digital man to
  delay (1000); // Waiting for one second to pass
  digitalWrite (13, LOW); // LOW Adjust the value of the 13 digital man to
  delay (1000); // Waiting for one second to pass
}
```

Notes et avertissements

Des jambes d'entrée similaires peuvent également être utilisées comme jambes numériques, alors appelées A0, A1, etc.

La fonction pinMode () dans l'Arduino

La fonction pinMode () prépare une jambe spécifique pour effectuer un comportement d'entrée ou de sortie. Voir le «Tutoriel des jambes numériques» pour plus d'informations sur les fonctions des jambes.

À partir de la version Arduino 1.0.1, il est devenu possible d'activer la résistance de levage interne en utilisant le mode INPUT_PULLUP. De plus, le mode INPUT désactive automatiquement les résistances de levage internes.

Structure générale

pinMode (broche, mode)

Transactions

épingle

Un numéro indiquant l'homme numérique à configurer en mode.

mode

Le mode qui détermine le comportement de la broche est INPUT, OUTPUT ou INPUT_PULLUP. Une explication complète du comportement et des fonctions des pieds numériques peut être trouvée sur cette page.

Valeurs renvoyées

Rien n'est retourné.

Exemples

Réglez le mode numérique 13 homme sur l'homme de sortie, puis inversez sa valeur de HIGH à LOW et vice versa toutes les secondes:

```
void setup ()
{
  pinMode (13, SORTIE); // 13 homme numérique l'a défini comme un homme est sorti
}


boucle vide ()
{
  digitalWrite (13, HAUT); // HIGH Ajuste la valeur de 13 digital man à
  retard (1000); // Attend une seconde pour passer
  digitalWrite (13, FAIBLE); // LOW Ajuste la valeur du 13 digital man à
  retard (1000); // Attend une seconde pour passer
}
```

Notes et avertissements

Des jambes d'entrée similaires peuvent également être utilisées comme jambes numériques, alors appelées A0, A1, etc.

Fonctions de temps dans Arduino

Les fonctions de cette section traitent des heures et des heures, par exemple pour créer des délais avec une certaine valeur requise par certaines tâches, ou pour exécuter un processus spécifique lorsqu'une période de temps spécifique s'est écoulée depuis le démarrage du tableau.

La fonction delay () dans l'Arduino

La fonction delay () interrompt l'exécution du programme pendant une période de temps spécifiée (en millisecondes).

Structure générale

retard (ms)

Transactions

SP

Un nombre long non signé représente le temps en millisecondes pendant lequel le programme est mis en pause.

Rappel: il y a 1 000 millisecondes par seconde.

Valeurs renvoyées

Rien n'est retourné.

Exemples

Utilisez la fonction delay () pour mettre le programme en pause pendant une seconde avant d'inverser le boîtier de sortie connecté à la lumière LED:

int ledPin = 13; // Lumière LED connectée à l'homme 13

```
void setup ()
{
  pinMode ( ledPin , OUTPUT); // Comme un homme a conduit, ledPin initialise l'homme
}

boucle vide ()
{
  digitalWrite ( ledPin , HIGH); // allumer la lumière
  retard (1000); // Attendez une seconde
  digitalWrite ( ledPin , LOW); // Éteindre la lumière
  retard (1000); // Attendez une seconde
}
```

Notes et avertissements

Bien qu'il soit très facile de créer une lumière LED clignotante avec la fonction delay () et que de nombreux codes utilisent cette fonction pour créer des délais requis par certaines tâches - comme les utiliser pour éviter le problème de rebond du bouton lorsqu'il est enfoncé (anti-rebond du bouton) - mais l'utilisation de la fonction delay () dans le code s'accompagne de nombreux inconvénients. Au cours de l'exécution du processus de retard lors de l' appel de la fonction, les valeurs des capteurs ne seront pas lus, les calculs ne seront pas effectués, et la condition de tout homme ne seront pas modifiés ou contrôlés ... etc., afin d' appeler cette fonction interrompra le reste des tâches et opérations et les arrêtera pendant un certain temps. Au lieu de cela, regardez la fonction millis () et l'exemple ci-dessous pour gérer les retards et contrôler efficacement la synchronisation. Les programmeurs expérimentés et compétents évitent

généralement d'utiliser la fonction delay () pour retarder les événements et contrôler leur minutage lorsque la période de retard dépasse quelques dizaines de millisecondes à moins que le code écrit ne soit très simple.

Certaines choses continuent de fonctionner pendant le contrôle delay () d' Atmega , comme les interruptions, car cela ne les perturbe pas. La communication série du récepteur RX est également enregistré, les valeurs de PWM (écrites en utilisant analogWrite ()) et l'état de la jambe sont conservés, et les interruptions continuent de travailler si besoin.

Fonction Arduino delayMicrosecondes ()

La fonction delayMicroseconds () interrompt l'exécution du programme pendant une période de temps spécifiée (en secondes). Actuellement, la valeur la plus élevée donnant un délai précis est 16383. Cette valeur peut changer dans les futures versions d'Arduino. La fonction delay () doit être utilisée à la place de cette fonction pour des retards de plus de quelques milliers de microsecondes.

Structure générale

delayMicrosecondes (us)

Transactions

nous

Un entier non signé représente le temps en microseconde auquel le programme est suspendu.

Rappel: il y a 1 000 microsecondes par milliseconde et 1 000 000 microsecondes toutes les 1 seconde.

Valeurs renvoyées

Rien n'est retourné.

Exemples

Le code suivant prépare le 8ème homme à agir comme un homme sortant. L'homme transmet alors une série d'impulsions d'une durée d'environ 100 microsecondes. Vous devez vous concentrer sur le mot «presque» car l'heure est absolument inexacte et cela est dû à

la perte de temps dans l'implémentation d'autres commandes et instructions dans le code:

```
int outPin = 8; // Label Man 8

void setup ()
{
  pinMode ( outPin , OUTPUT); // Comme un homme est sorti, Pin crée l'homme
}

boucle vide ()
{
  digitalWrite ( outPin , HIGH); // Rend l'état de l'homme élevé
  delayMicrosecondes (50); // Pause de 50 microsecondes
  digitalWrite ( outPin , LOW); // Rend l'état de l'homme bas
  delayMicrosecondes (50); // Pause de 50 microsecondes
}
```

Notes et avertissements

Cette fonction fonctionne avec une précision absolue dans la plage comprise entre 3 microsecondes et plus. La fonction delayMicroseconds () ne peut pas être garantie de fonctionner avec précision pour des délais inférieurs à cette valeur.

À partir de la version 0018, la fonction delayMicroseconds () ne désactive plus les interruptions.

Fonction Arduino micros ()

La fonction micros () renvoie le temps d'exécution de la carte Arduino du programme en cours en microsecondes.

La valeur calculée par cette fonction atteindra l'état d'éruption cutanée (c'est-à-dire qu'elle reviendra à zéro) après environ 70 minutes. Sur les cartes Arduino 16 MHz (telles que Duemilanove et Nano), la précision de cette fonction est de 4 microsecondes (c'est-à-dire que la valeur renvoyée par la fonction est toujours un multiple de 4). Sur les cartes Arduino fonctionnant à 8 MHz (comme LilyPad), la précision de cette fonction est de 8 microsecondes.

Structure générale

temps = micros ()

Valeurs renvoyées

Renvoie un nombre long non signé représentant l'heure de démarrage de la carte Arduino du programme en cours jusqu'au moment où la fonction micros () est appelée.

Exemples

Utilisez la fonction micros () pour savoir combien de temps la carte Arduino a démarré:

```
unsigned long time;


void setup () {
  Serial.begin (9600);
}
void loop () {
  Serial.print ("Time:");
  time = micros ();
```

```
  Serial.println (time); // Print the duration of the program since it started working
  delay (1000); // Wait for one second not to send a huge amount of data
}
```

Notes et avertissements

N'oubliez pas: il y a 1 000 microsecondes par milliseconde et 1 000 000 microsecondes par seconde.

Fonction Arduino millis ()

La fonction millis () renvoie le temps d'exécution de la carte Arduino pour le programme en cours en millisecondes.

La valeur que cette fonction calcule atteindra l'état de débordement (c'est-à-dire qu'elle reviendra à zéro) après environ 50 jours.

Structure générale

time = millis ()

Valeurs renvoyées

Un nombre long non signé représentant l'heure de début de la carte Arduino du programme en cours est renvoyé au moment où la fonction millis () est appelée en millisecondes.

Exemples

Utilisez la fonction millis () pour savoir depuis quand la carte Arduino a commencé à fonctionner:

non signé depuis longtemps;

```
void setup () {
  Serial.begin (9600);
}
boucle void () {
  Serial.print ("Heure:");
  temps = millis ();

  Serial.println (heure); // Affiche la durée du programme depuis qu'il a commencé à fonctionner
  retard (1000); // Attendez une seconde pour ne pas envoyer une énorme quantité de données
}
```

Notes et avertissements

Veuillez noter que le type de valeur retourné par la fonction millis () est un nombre long non signé, donc des erreurs logiques peuvent se produire si le programmeur essaie d'effectuer des opérations mathématiques avec des types de nombres plus petits, tels que int. Même les calculs avec un nombre long signé peuvent entraîner des erreurs, car sa capacité maximale est la moitié de celle du numéro non référencé correspondant.

Fonctions mathématiques dans Arduino

Les fonctions mathématiques traitent généralement des nombres, car elles effectuent diverses opérations mathématiques sur eux, telles que le calcul d'une valeur absolue, d'une racine de nombre, d'un sinus ou d'une génération de nombres aléatoires.

() Abdos

La fonction abs () calcule la valeur absolue du nombre qui lui est passé.

Contraindre ()

La fonction constrain () limite un nombre dans un champ spécifié.

() Cos

La fonction cos () calcule l'angle qui lui est passé.

() Carte

La fonction map () repositionne un nombre du domaine dans lequel il se trouve vers un autre domaine.

() Max

La fonction max () renvoie la plus grande valeur qui lui est passée.

() Min

Min () renvoie la plus petite valeur qui lui a été transmise.

() Pow

La fonction pow () calcule la valeur yx , qui soulève la base Y à la puissance x des deux valeurs qui sont transmises à elle.

Aléatoire ()

La fonction random () génère et renvoie un nombre pseudo-aléatoire.

() Aléatoire

La fonction randomSeed () crée la graine que le générateur de nombres pseudo-aléatoires utilise dans le processus de génération.

() Péché

La fonction sin () calcule le sinus de l'angle passé à.

() Carré

La fonction sq () calcule le nombre de carrés (multiplié par le nombre lui-même) et le renvoie.

Sqrt ()

La fonction sqrt () calcule la racine carrée du nombre qui lui est passé.

() Bronzer

La fonction tan () calcule la tangente de l'angle passé à.

Manipulation et vérification des polices dans Arduino

Ces fonctions fonctionnent généralement avec des caractères, vérifiant qu'un caractère particulier appartient à un ensemble particulier de caractères.

isAlpha ()

La fonction isAlpha () vérifie si le caractère passé est un caractère alphabétique.

isAlphaNumeric ()

La fonction isAlphaNumeric () vérifie si le caractère qui lui est passé est un caractère alphabétique ou un nombre.

isAscii ()

La fonction isAscii () vérifie si le caractère qui lui est passé est l'un des caractères Ascii.

isControl ()

La fonction isControl () vérifie si le caractère passé est un caractère de contrôle.

isDigit ()

La fonction isDigit () vérifie si le caractère passé est un nombre.

isGraph ()

La fonction isGraph () vérifie si la police qui lui est transmise est imprimable et contient du contenu visible (le blanc est imprimable mais n'a pas de contenu visuel).

isHexadecimalDigit ()

IsHexadecimalDigit () vérifie si le caractère passé est un chiffre hexadécimal (chiffre hexadécimal); c'est-à-dire qu'il se situe entre AF ou 9-0.

isLowerCase ()

La fonction isLowerCase () vérifie si le caractère qui lui est passé reçoit un caractère alphabétique minuscule .

isPrintable ()

IsPrintable () vérifie si le caractère passé est imprimable (c'est-à-dire l'un des caractères qui produit une sortie même s'il est vide).

isPunct ()

La fonction isPunct () vérifie si la police qui lui est passée est un signe de ponctuation (c'est-à-dire est la virgule, le point-virgule, le point d'exclamation, etc.)?

isSpace ()

La fonction isSpace () vérifie si le caractère qui lui est passé est un espace.

isUpperCase ()

La fonction isUpperCase () vérifie si le caractère qui lui est passé est un caractère alphabétique minuscule .

isWhitespace ()

La fonction isWhitespace () vérifie si le caractère qui lui est passé est un espace blanc.

Fonctions de conversion Arduino

Les fonctions de conversion convertissent entre différents types de données dans Arduino.

octet ()

La fonction byte () convertit la valeur qui lui est passée de type octet.

char ()

La fonction char () convertit la valeur passée en type char.

flottant ()

La fonction float () convertit la valeur qui lui est passée (float).

int ()

La fonction int () convertit la valeur transmise en type entier.

longue ()

La fonction long () convertit la valeur passée au type «long».

mot ()

La fonction word () convertit la valeur transmise en type de mot.

Fonctions de contrôle provinciales dans Arduino

Les fonctions de cette section contrôlent les districts dans Arduino, activant ou désactivant une interruption spécifique sur une jambe, ou activant ou désactivant toutes les interruptions.

Attachinterrupt ()

La fonction attachInterrupt () active une interruption externe spécifique sur l'une des jambes de la carte arduino .

() Détacherinterrompre

La fonction detachInterrupt () désactive une interruption externe spécifique associée à l'une des jambes de la carte Arduino.

() Interruptions

La fonction interrupts () désactive les interruptions après l'avoir arrêtée en appelant la fonction nointerrupts ().

() Pas d'interruptions

La fonction noInterrupts () désactive toutes les interruptions de la carte Arduino.

Chapitre VI

Outils supplémentaires

Le programme de mots clés dans Arduino

Le mot-clé PROGMEM est utilisé avec des variables lorsque ses données doivent être stockées dans la mémoire programme (mémoire flash) plutôt que dans la SRAM. Pour plus d'informations sur les types de mémoire disponibles dans Arduino, consultez cette page.

Le mot-clé PROGMEM est classé sous «qualificatif de variable» et ne doit être utilisé qu'avec les types de données définis dans la bibliothèque pgmspace.h . Ce mot indique au banquier de placer les données variables que vous avez utilisées avec lui en flash au lieu de dans la SRAM où elles doivent être stockées dans l'état normal.

Le mot clé PROGMEM fait partie de la bibliothèque pgmspace.h . Ce disque est automatiquement inclus dans les versions récentes de l'IDE Arduino. Cependant, si vous utilisez une version antérieure à 1.0 (2011), la bibliothèque doit être incluse manuellement au début de votre code en utilisant include #:

#include < avr / pgmspace.h >

Structure générale

const dataType variableName [] PROGMEM = {data0, data1, data3…};

DataType représente tout type de données et variableName est le nom du tableau de données à stocker dans la mémoire flash. Sachez que le mot clé PROGMEM étant contraint à une variable, il n'y a pas de règle qui l'oblige à être à un endroit spécifique lors de la déclaration d'une variable; Cependant, certains tests ont suggéré que PROGMEM, dans différentes versions d'Arduino (qui utilise le

Gulf Banker), peut fonctionner dans un emplacement spécifique uniquement lors de la déclaration d'une variable. Le deuxième exemple est essayé dans la section exemples de la version Arduino 13; l'exemple peut mieux fonctionner dans les versions antérieures si PROGMEM est placé après le nom de la variable.

const dataType variableName [] PROGMEM = {};

const PROGMEM dataType variableName [] = {};

const dataType PROGMEM variableName [] = {};

Puisqu'il est possible d'utiliser le mot-clé PROGMEM avec une variable, il est préférable de l'utiliser avec un tableau (ou toute structure similaire en C pour stocker des données, qu'elles soient mentionnées dans cette documentation ou non) pour contenir les données à stocker en flash mémoire si ces données sont volumineuses.

L'utilisation de PROGMEM nécessite une procédure en deux étapes. Après avoir stocké les données dans la mémoire flash (la première étape), des fonctions spéciales (fonctions) définies dans la bibliothèque pgmspace.h doivent être utilisées afin de lire ces données depuis la mémoire du programme et de les mettre dans la SRAM, afin que nous puissions prendre profiter de ces données et les utiliser.

Exemples

Le code suivant montre comment écrire des données dans la mémoire flash, puis les relire:

```
// Mémorise quelques nombres sans entier
const PROGMEM uint16_t charSet [] = {65000, 32796, 16843, 10, 11234};


// Sauvegarde quelques caractères
const char signMessage [] PROGMEM = {"JE SUIS UN PRÉDATEUR, UN COMBATANT INVISIBLE. CRÉÉ PAR LE DÉPARTEMENT DES ÉTATS-UNIS"};


unsigned int displayInt ;
int k; // variable de compteur
char myChar ;



void setup () {
  Serial.begin (9600);
  while (! Série); // En attente de connexion au port série


  // Tapez ici tout ce que vous voulez configurer et exécutez-le une fois
  // Relire les nombres entiers
  pour (k = 0; k <5; k ++)
```

```
  {
    displayInt = pgm_read_word_near ( charSet + k);
    Serial.println ( displayInt );
  }
  Serial.println ();

  // Relire les caractères
  pour (k = 0; k < strlen_P ( signMessage ); k ++)
  {
    myChar = pgm_read_byte_near ( signMessage + k);
    Serial.print ( myChar );
  }

  Serial.println ();
}

boucle void () {
  // Écrivez ici votre programme pour l'exécuter fréquemment


}
```

Lorsque vous travaillez avec une grande quantité de texte, comme avoir un projet sur l'écran LCD, il est judicieux de créer et de configurer un tableau de chaînes de texte pour les stocker. Puisque les chaînes de texte elles-mêmes sont des tableaux, une matrice 2D doit être utilisée. L'exemple suivant illustre l'idée de placer de grandes données dans la mémoire du programme:

```
/ *
 Démo de chaînes PROGMEM
```

Comment stocker une table de chaînes dans la mémoire programme (flash),

et récupérez-les.

Informations résumées de:

http://www.nongnu.org/avr-libc/user-manual/pgmspace.html

Mettre en place une table (tableau) de chaînes dans la mémoire programme est légèrement compliqué, mais

voici un bon modèle à suivre.

La configuration des chaînes est un processus en deux étapes. Définissez d'abord les chaînes.

* /

```
#include <avr / pgmspace.h>

const char string_0 [] PROGMEM = "String 0"; // is the string to store. Others to suit your "String 0" series

const char string_1 [] PROGMEM = "String 1";

const char string_2 [] PROGMEM = "String 2";

const char string_3 [] PROGMEM = "String 3";

const char string_4 [] PROGMEM = "String 4";

const char string_5 [] PROGMEM = "String 5";



// Then create a table that points to these strings
```

```
const char * const string_table [] PROGMEM = {string_0, string_1,
string_2, string_3, string_4, string_5};


char buffer [30]; // Make sure that this matrix is   large enough to
accommodate the largest text string


void setup ()
{
  Serial.begin (9600);
  while (! Serial); // Wait for the serial port to connect
  Serial.println ("OK");
}



void loop ()
{
  / * Requires the use of a strings table in memory
     Use special functions to retrieve data
      A text string from the strcpy_P program memory copies the
function
      (In our case buffer is the variable memory)
      Big enough RAM Make sure that the portion is reserved in
memory
      To recover data from program memory
      * /
```

```
  for (int i = 0; i <6, i ++)
  {
    strcpy_P (buffer, (char *) pgm_read_word (& (string_table [i]))); //
Necessary casts and dereferencing, just copy.
    Serial.println (buffer);
    delay (500);
  }
}
```

Notes et avertissements

Veuillez noter que les variables doivent être soit des variables générales soit des variables locales statiques (définies à l'aide du mot-clé static) pour pouvoir utiliser le mot PROGMEM avec lui.

L'exemple suivant ne fonctionnera pas lorsqu'il est placé dans une variable:

```
const char long_str [] PROGMEM = "Bonjour, je voudrais vous parler un peu de moi. \ n";
```

Alors que la déclaration suivante fonctionnera pour une variable si elle est définie localement dans une fonction:

```
const static char long_str [] PROGMEM = "Bonjour, je voudrais vous parler un peu de moi. \ n"
```

Appel prolongé F ()

Quand quelque chose comme:

Serial.print ("Ecrire quelque chose sur le moniteur série");

La chaîne de texte à imprimer normalement stockée dans la RAM. Si votre code imprime beaucoup de scripts sur le port série, il remplira la mémoire. Si vous disposez d'un espace flash, vous pouvez facilement l'investir en y stockant ces textes en utilisant la formule suivante:

Serial.print (F ("Ecrire quelque chose sur le moniteur série qui est stocké dans FLASH"));

Chapitre VII

Proteus

Dans ce chapitre, nous réaliserons la première expérience, qui consiste à allumer et éteindre la LED à plusieurs reprises. Pour nous assurer que tous les concepts et étapes sont liés les uns aux autres,

nous expliquerons toutes les étapes à partir de la création d'un projet sur Proteus jusqu'à l'extraction d'un fichier Hex qui sera utilisé pour programmer Arduino en plus d'exécuter l'expérience et d'observer les résultats. . En outre, l'explication étape par étape sera couverte.

Conception de cercle sur les protéas

1. Ouvrez Proteus 8 et choisissez un nouveau projet.

2. Tapez le nom du projet et l'emplacement de classement, puis cliquez sur Suivant.

3. Choisissez Créer un schéma à partir de modèles, puis choisissez un modèle (par défaut).

. Choisissez ne pas créer de pcb comme indiqué, puis choisissez Suivant.

5. Bien qu'il existe une plate-forme sur Proteus pour écrire et implémenter des codes Arduino, je préfère utiliser le programme Arduino principal, alors choisissez l'option spécifiée de la manière suivante.

6. Ceci est le résumé du projet , nous créerons uniquement un schéma. Après avoir appuyé sur Terminer, nous serons prêts à placer les appareils et à les connecter ensemble.

7. Nous choisirons trois appareils, à savoir

Arduino UNO

Résistance de 220 ohms

LED ROUGE

Les étapes suivantes expliquent comment choisir l'UNO et le placer sur la liste des périphériques qui seront utilisés dans la conception. De la même manière, choisissez la LED ROUGE et la résistance 220ohm.

9. L'exemple suivant montre comment placer l'Arduino sur l'espace schématique. De la même manière, suivez les étapes suivantes et mettez la résistance et la LED.

10. Après avoir placé tous les appareils sur l' asmatic et connecté les extrémités, nous aurons la forme suivante.

. Que manque-t-il maintenant jusqu'à ce que le cercle soit terminé? Il ne fait aucun doute que c'est la Terre. Pour choisir le sol, suivez les étapes illustrées ci-dessous. Ainsi, nous avons terminé les réglages de connexion pour le circuit sur le programme protéine.

Remarque:

La carte Arduino n'a pas besoin d'être connectée à l'alimentation ou même à la terre, car Proteus effectue ce processus implicitement pour vous.

Ecrire le code sur le programme Arduino

1. Ouvrez le programme Arduino, ouvrez un nouveau projet, écrivez le code suivant, puis enregistrez le projet sous le nom E1_LED_Blinking

```
const int LED_Pin = 13;

void setup () {
  // put your setup code here, to run once:
  pinMode (LED_Pin, OUTPUT);
}

void loop () {
  // put your main code here, to run repeatedly:
  digitalWrite (LED_Pin, HIGH);
  delay (1000);
  digitalWrite (LED_Pin, LOW);
```

```
  delay (1000);
}
```

Explication du code:

Le code commence par annoncer les variables qui sont utilisées dans plus d'une portée du programme. Le "Scope" est le terme entre parenthèses de type {}. Nous avons donc commencé par annoncer la variable LED_Pin qui était utilisée dans le setup et la boucle. La commande const est utilisée pour garantir que la valeur de la variable n'est pas modifiée à nouveau, et comme la variable LED_Pin conserve la valeur du numéro de port, elle ne doit plus être modifiée. Nous notons que la valeur 13 a été affectée à la variable LED_Pin , car nous avons connecté la LED au port numéro 13. Dans la configuration, le type de port (port) sera défini s'il est sorti ou entré via la fonction PinMode . À l'intérieur de la boucle, le code qui se répète en continu sera écrit en continu. Le code à l'intérieur de la boucle commence par la ligne (digitalWrite (LED_Pin , HIGH), ce qui signifie une sortie de 5 volts vers le port appelé LED_Pin , après quoi nous remarquons la ligne de retard (1000), ce qui signifie que le code restera le même pendant un période de 1 seconde, on remarque alors la présence de la ligne (digitalWrite) LED_Pin , LOW à travers laquelle 5 volts seront empêchés de passer au port 13 et pour assurer la continuité de la LED s'arrête d'allumer pendant une période de 1 seconde il fallait mettre (délai) 1000 à la fin de la boucle également.Si à travers le code à l'intérieur de la boucle, il s'allumera LED pendant une seconde et il s'éteindra pendant une seconde et ainsi il continuera sans s'arrêter.

2. Vérifiez le code en appuyant sur le bouton indiqué sur la figure pour vous assurer que le code ne contient aucune erreur.

3. Si le code est correct, vous verrez un message similaire au message suivant sous le programme Arduino.

4. Après vous être assuré que le code est exempt d'erreurs, accédez au menu d'esquisse et choisissez Exporter le binaire compilé. Il s'agit d'extraire le fichier hexadécimal. Qui est utilisé pour programmer les microcontrôleurs.

. En allant dans le dossier dans lequel le projet a été enregistré, vous trouverez un fichier .hx . Avec le même nom que le projet.

6. Revenez au cercle dans Proteus, appuyez sur Arduino pour ajouter le fichier de script comme indiqué dans les étapes suivantes.

7. Allumez le circuit et remarquez comment la LED s'allume puis s'éteint à plusieurs reprises.

8. Essayez de changer le temps de retard à 500 au lieu de 1000, effectuez les étapes 2 à 7 et notez la différence.

Débat

Supposons que vous écriviez le code dans l'image suivante et que vous ignoriez le délai après le digitalWrite (LED_Pin , LOW). Que

pensez-vous qu'il se passe dans ce cas?

```
const int LED_Pin = 13;

void setup () {
  // put your setup code here, to run once:
  pinMode (LED_Pin, OUTPUT);
}

void loop () {
  // put your main code here, to run repeatedly:
  digitalWrite (LED_Pin, HIGH);
  delay (1000);
  digitalWrite (LED_Pin, LOW);
}
```

Alors , laissez - nous faire le processus de mise en œuvre, nous partirons de la boucle et nous allons mettre 5 volts sur le port appelé LED_Pin qui est à l' origine numéro de port 13. Ensuite , nous allons passer à la deuxième ligne pour la mettre en œuvre et nous trouver retard de 1000 et que les offres de la fonction de retard avec les valeurs en millisecondes , alors ce cela signifie que le délai se poursuivra pendant une période de 1 seconde, et ce délai restera le dernier état , le micro était, donc sera allumé la LED pour une période de 1 seconde, c'est-à-dire jusqu'à la fin de la commande de retard. Maintenant, nous allons exécuter la troisième ligne qui contient la commande (digitalWrite (LED_Pin , LOW) dans laquelle la LED sera éteinte. Eh bien la LED s'est éteinte maintenant mais alors quoi? Concentrez-vous sur moi bien ici nous sommes à l'intérieur de la boucle et donc le code recommencera à s'exécuter à

partir de la première ligne A à l'intérieur de la boucle dans laquelle la commande qui émettra 5 volts vers la LED et elle s'allumera à nouveau! Voici le problème où la dernière ligne sera exécutée et immédiatement après elle la première ligne sera exécutée et l'œil ne pourra pas remarquer que la LED est éteinte car l'implémentation du code est calculée en microsecondes pour cela Le délai devait être mis après la commande qui exigeait que la LED soit éteindre.

Chapitre VIII

Matrices et boucles pour et condition pour contrôler les LED

Dans cette leçon, nous contrôlerons quatre lumières LED, chaque lampe sera contrôlée par bouton poussoir. Ce que nous allons apprendre ici, c'est comment mettre les ports connectés aux LED dans une variable est un tableau, et la même chose pour les commutateurs sera également placé dans un tableau. De cette façon, le code sera raccourci et plus professionnel. La leçon explique également comment utiliser les boucles for pour accéder aux éléments d'un tableau et aux valeurs qui y sont enregistrées. En outre, il expliquera comment exécuter une commande en fonction de la satisfaction d'une condition spécifique.

Conception de cercle sur Proteus

Ouvrez Proteus et concevez le cercle comme indiqué dans la figure suivante

Explication du circuit

Quatre boutons-poussoirs ont été connectés aux ports Arduino du port numéro 1 au port numéro 4, les ports ont été configurés pour contenir en interne une résistance de pull-up afin qu'il n'y ait pas besoin de connecter des résistances ou des sources de tension avec les boutons-poussoirs et donc il sera travailler à travers la logique inverse Lorsqu'un bouton spécifique est enfoncé, ce port lira la logique LOW, et lorsqu'un bouton n'est pas enfoncé, ce port lira la logique HIGH et c'est ce qui devrait être considéré dans le code (vous pouvez consulter la leçon Sixièmement, il existe une expérience complète pour comprendre cette méthode). Pour les LED, ils sont connectés aux ports 9 à 12 via une résistance de 220 ohms utilisée pour réduire le courant. Dans le code, nous ferons en sorte que chaque bouton n'illumine qu'une seule LED.

Ecrire le code sur le programme Arduino

Écrivez le code suivant sur le programme Arduino, puis extrayez le fichier Hex et programmez l'Arduino comme décrit dans la leçon 4.

```
int pushButtons [4] = {1,2,3,4};
int ledPins [4] = {9,10,11,12};



void setup () {
  // put your setup code here, to run once:
  for (int i = 0; i <4; i ++)
  {
    pinMode (pushButtons [i], INPUT);
    pinMode (ledPins [i], OUTPUT);
    digitalWrite (pushButtons [i], HIGH); // activate pullup resistor
  }
}

void loop () {
  // put your main code here, to run repeatedly:
  for (int i = 0; i <4; i ++)
```

```
  {
    int val = digitalRead (pushButtons [i]);
    if (val == 0)
      digitalWrite (ledPins [i], HIGH);
    else
      digitalWrite (ledPins [i], LOW);
  }
}
```

Explication du code

Le code commence par définir des variables globales qui fonctionneront dans les champs de configuration et de boucle. Nous avons défini deux matrices de type entier , chaque matrice est constituée de quatre éléments. Dans la première matrice, appelée pushButtons , les numéros de port connectés aux pushButtons sont attribués, et ce sont les ports 1 à 4. De même, pour la deuxième matrice, qui est appelée ledPins , les numéros de port connectés aux LED ont été placés à l'intérieur Le tableau, qui comprend les ports 9 à 12. Nous devons noter qu'après le nom du tableau, le nombre d'éléments du tableau est entre parenthèses telles que [4].

```
int pushButtons [4] = {1,2,3,4};
int ledPins [4] = {9,10,11,12};
```

Quant au code à l'intérieur de la zone de configuration, comme nous l'avons appris précédemment, il est souvent utilisé pour définir la position des ports afin qu'ils soient à l'état d'entrée ou de sortie. Mais avant d'expliquer le code dans le champ de configuration, nous allons d'abord discuter de certains des concepts de base liés aux matrices.

La numérotation des éléments à l'intérieur des matrices commence par le numéro 0, par exemple le premier élément du tableau ledPins est [0] ledPins qui contient la valeur 9 et de cela nous concluons que le deuxième élément du tableau est l'élément numéro 1 et bientôt. Donc [0] ledPins est une variable qui contient la valeur 9, et si nous l'appelons à tout moment, cela signifie que nous avons appelé la valeur à l'intérieur qui est 9 à moins qu'elle ne soit modifiée. Comme le port

numéro 9 est connecté à une LED, il doit être configuré pour fonctionner comme un port de sortie, le code suivant explique cette étape:

pinMode (ledPins [0], OUTPUT);

Notez que le nombre 0 est à l'intérieur de la matrice ledPins , qui indique l'élément zéro, qui a une valeur de 9. Nous voulons maintenant définir tous les ports de 9 à 12 afin de fonctionner comme des ports de sortie car ils sont connectés avec des LED. Nous pouvons le faire via le code suivant dans la fonction de configuration.

```
void setup () {
   pinMode (ledPins [0], OUTPUT);
   pinMode (ledPins [1], OUTPUT);
   pinMode (ledPins [2], OUTPUT);
   pinMode (ledPins [3], OUTPUT);
}
```

Dans le code ci-dessus, nous avons appelé la fonction pinMode sur chaque ligne pour que les ports fonctionnent soit dans le cas de la sortie soit de l'entrée, l'entrée (Arguments) qui sera passée à la fonction est le numéro de port et l'état qui doit travailler dessus (entrée / sortie), et puisque nous traitons les ports connectés avec des LED, l'état sera toujours la sortie. Quant aux numéros de port, ils étaient précédemment définis dans le tableau ledPins , donc sur chaque ligne le nom du tableau avec le numéro d'élément sera placé à l'intérieur de la fonction pinMode . Ce code est vrai à cent pour cent, mais il manque de professionnalisme et explique que son rédacteur est un peu faible en programmation, mais pourquoi? Imaginez si le nombre de ports à l'intérieur du tableau est de 10, cela signifie que nous écrirons 10 lignes pour définir le type de port qui servira de port de sortie. Mais quelle est la manière professionnelle? Il est préférable d'utiliser des boucles itératives pour réduire le code et le rendre plus professionnel et flexible. Nous utiliserons la boucle for comme indiqué dans le code suivant.

```
for (int i = 0; i <4; i ++)
  {
```

```
    pinMode (ledPins [i], OUTPUT);
  }
```

Dans le code ci-dessus, nous avons utilisé la boucle for et dans la définition de for, nous avons défini la variable i , qui est de type entier, et nous lui avons attribué la valeur initiale 0 car le premier élément du tableau commence par zéro. Maintenant, implémentons la boucle for ensemble. Dans le premier cycle, la valeur de i sera égale à 0. Cette valeur sera compensée à l'intérieur du code à l'intérieur du champ de boucle for, donc le code sera comme suit (pinMode (ledPins [0], OUTPUT), après avoir exécuté tout le code à l'intérieur du champ de la boucle for Le for implémentera le code partiel qui affectera la valeur de i et dans ce code est ++ i , ce qui signifie que la variable i deviendra sa valeur égale à un, maintenant le for testera la condition i <4 et certainement la condition sera remplie car la valeur de i est maintenant un et elle est en fait inférieure à 4. Et puisque la condition est remplie, nous pouvons maintenant exécuter ce qui se trouve à l'intérieur de la boucle for, et cette fois faire n'oubliez pas que la valeur de i est devenue un, donc la variable sera compensée. Pour entrer pin pour devenir le pinMode (ledPins [1], code OUTPUT, et en exécutant le port numéro dix, il sera mis dans l'état de sortie. travail.

De la même manière, nous pouvons définir la matrice des ports d'entrée qui s'étendent du port numéro 1 à 4, et le code pour eux peut être ajouté à la boucle for à l'intérieur du réglage des ports de sortie. Il est très important de noter que les numéros de port 1 à 4 sont des nombres à l'intérieur du tableau tandis que le tableau lui-même commence à partir de l'élément 0 et se terminera par l'élément 3 car il a quatre éléments.

```
void setup () {
  for (int i = 0; i <4; i ++)
  {
    pinMode (pushButtons [i], INPUT);
    pinMode (ledPins [i], OUTPUT);
  }
}
```

Enfin, nous ajouterons un code partiel qui rendra les ports d'entrée liés à la résistance de tirage interne à la boucle itérative où il fonctionnera avec le même concept précédent.

```
void setup () {
  for (int i = 0; i <4; i ++)
  {
    pinMode (pushButtons [i], INPUT);
    pinMode (ledPins [i], OUTPUT);
    digitalWrite (pushButtons [i], HIGH); // activate pullup resistor
  }
}
```

Jusqu'à présent, nous avons terminé le code pour définir le mode de port, mais qu'en est-il du code à l'intérieur de la fonction de boucle? Comme vous le savez, le code de la boucle est le code responsable de la mise en œuvre de l'idée de base.

Le code à l'intérieur de la fonction de boucle est un code qui est exécuté en continu jusqu'à l'infini. Lorsque vous atteignez la dernière ligne de la fonction, la fonction recommence à fonctionner à partir de la première ligne. Il est à noter ici que le microcontrôleur de la carte Arduino Uno est ATMEGA328P connecté à une source d'horloge fonctionnant à 16 MHz et que presque le microcontrôleur effectue une opération par impulsion d'horloge, c'est-à-dire que le microcontrôleur peut implémenter 16 millions d'instructions par seconde! J'ai délibérément mentionné cette information maintenant parce que le code de la boucle dans cette leçon dépend de cette fonctionnalité et vous ne pourrez pas le comprendre tant que vous ne ressentirez pas la vitesse à laquelle le microcontrôleur fonctionne.

```
void loop () {
  // put your main code here, to run repeatedly:
  for (int i = 0; i <4; i ++)
  {
```

```
    int val = digitalRead (pushButtons [i]);
    if (val == 0)
      digitalWrite (ledPins [i], HIGH);
    else
      digitalWrite (ledPins [i], LOW);
  }
}
```

Le code commence par une boucle for qui va de 0 à 3, vous commencerez par placer la valeur 0 dans la variable i et le code à l'intérieur de la boucle sera exécuté. Dans la première ligne à l'intérieur de la boucle, nous avons défini la variable val de type entier, dans laquelle la valeur du port 1 sera lue, c'est-à-dire [bouton-poussoir [0], si le bouton-poussoir est enfoncé, la fonction digitalRead lira le valeur 0 et dans le cas contraire, il lira la valeur 1, alors ce sera mis La valeur dans la variable val. Ensuite, le microcontrôleur implémentera la fonction de condition if else et testera la condition si la variable val contient la valeur 0 ou LOW alors si la condition est remplie la valeur de HIGH sera placée sur le port associé à la variable [ledPins [0 and si la condition n'est pas remplie la valeur LOW sera placée sur le port associé à la variable [ledPins [0 Nous avons donc exécuté la première boucle itérative dans laquelle la valeur de la variable i était égale à zéro. La fonction continuera à fonctionner et chaque fois que vous augmentez la valeur de i de un, puis exécutez le code avec la nouvelle valeur de i jusqu'à ce qu'il devienne i égal à 3, le code est exécuté en fonction de cette valeur, puis de la valeur de i revient à 0.

Supposons maintenant que vous ayez implémenté le code et que vous l'ayez chargé dans l'Arduino, puis que vous avez appuyé sur le bouton connecté au port numéro 3 situé dans [pushbutton [2. En fait, le code dans la fonction de boucle testera tous les boutons poussoirs du [bouton-poussoir [0 pour atteindre [bouton-poussoir] 3 mais cela se fera très rapidement et à chaque fois qu'il passera le test de la condition liée au [bouton-poussoir [2] qui va allumez la LED connectée au port numéro 11.